

Large Language Models: An Introduction

Robert Haase

These slides can be reused under the terms of the [CC-BY4.0](https://creativecommons.org/licenses/by/4.0/) license.

Quiz

What is **this part of a DNN** typically called?

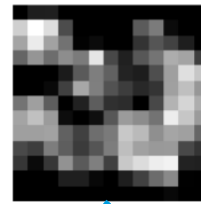
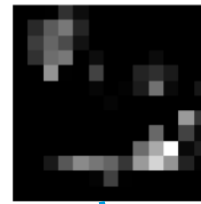
Layer 1 (256, 100, 100)



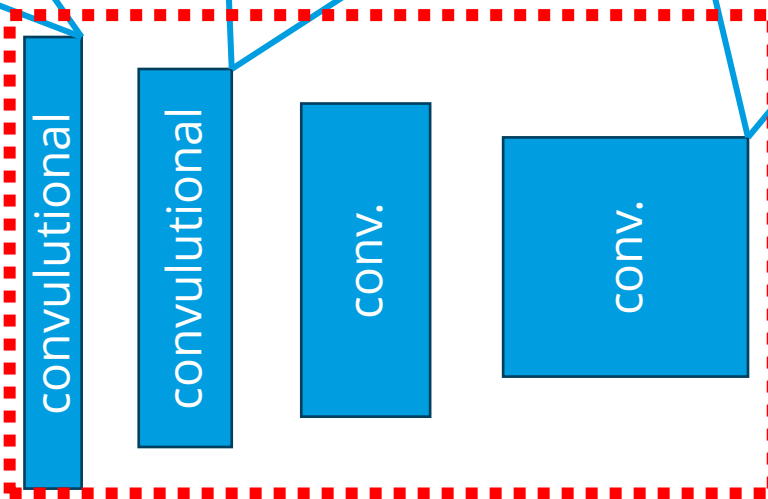
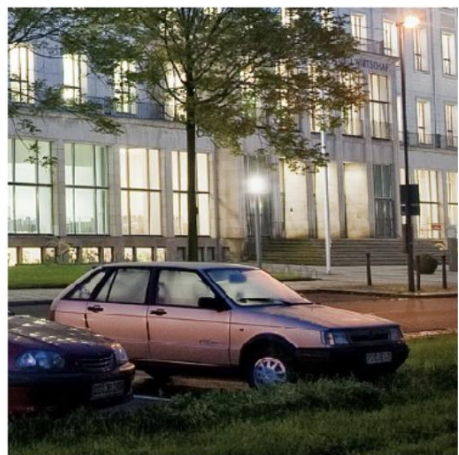
Layer 2 (512, 50, 50)



Layer 4 (2048, 13, 13)



400x400



- Beach wagon
- goldfish
- palace

Reducer



Increaser



Encoder

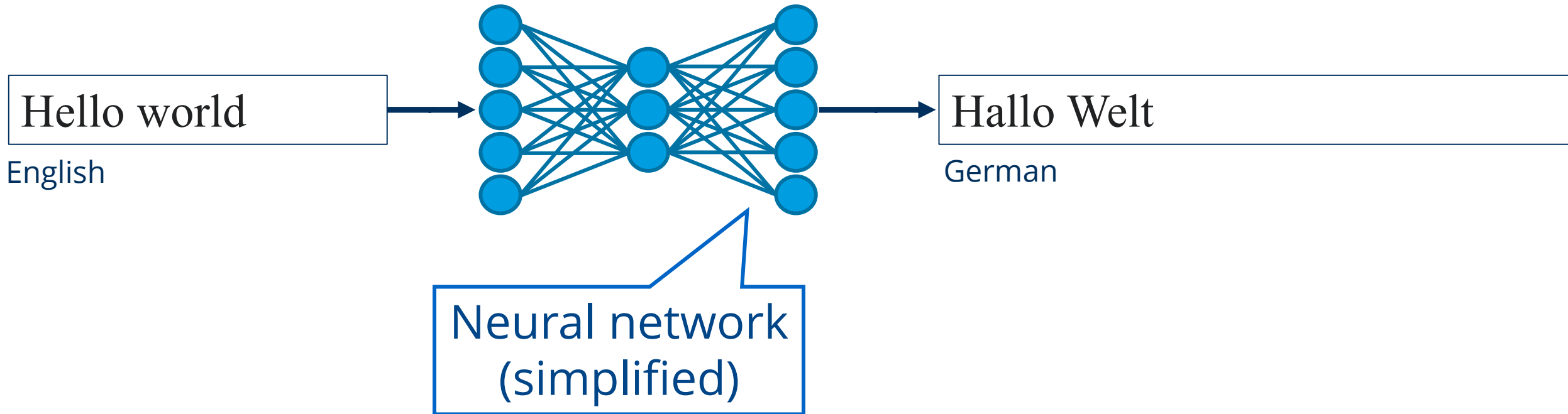


Decoder



Large Language Models (LLMs)

Text-to-text, translation



Translating training materials



Multidimensional image stacks

Multidimensional image data can be handled in a similar way as multi-channel image data.

Three-dimensional image stacks

There are also images with three spatial dimensions: X, Y, and Z. You find typical examples in microscopy and in medical imaging. Let's take a look at an Magnetic Resonance Imaging (MRI) data set:

```
[1]: from skimage.io import imread
from stackview import imshow
import matplotlib.pyplot as plt
image_stack = imread('.../data/Haase_MRT_tf13d1.tif')
```

Image slicing

As all three dimensions are spatial dimensions, we can also make slices orthogonal to the image plane and corresponding to Anatomical planes. To orient the images correctly, we can transpose their axes by adding `...T` by the end.

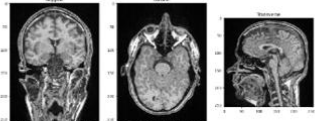
```
[2]: sagittal = image_stack[:, :, 128].T
coronal = image_stack[:, 128, :].T
transverse = image_stack[96]

fig, axs = plt.subplots(1, 3, figsize=(15, 15))

# show orthogonal planes
axs[0].imshow(sagittal, cmap='Greys_r')
axs[0].set_title('Sagittal')

axs[1].imshow(coronal, cmap='Greys_r')
axs[1].set_title('Coronal')

axs[2].imshow(transverse, cmap='Greys_r')
axs[2].set_title('Transverse');
```



Videos

If an image dataset has a temporal dimension, we call it a video. Processing videos works similar to multi-channel images and image stacks. Let's open a microscopy dataset showing yeast



Multidimensional Bildstapel

Multidimensionale Bilddaten können ähnlich wie mehrkanalige Bilddaten behandelt werden.

Dreidimensionale Bildstapel

Es gibt auch Bilder mit drei räumlichen Dimensionen: X, Y und Z. Typische Beispiele finden sich in der Mikroskopie und in der medizinischen Bildgebung. Schauen wir uns ein Magnetresonanztomographie (MRT) Datensatz an:

```
[1]: from skimage.io import imread
from stackview import imshow
import matplotlib.pyplot as plt
image_stack = imread('.../data/Haase_MRT_tf13d1.tif')
```

Bildschnitte

Da alle drei Dimensionen räumliche Dimensionen sind, können wir auch Schnitte orthogonal zur Bildebene machen, die den Anatomischen Ebenen entsprechen. Um die Bilder korrekt zu orientieren, können wir ihre Achsen transponieren, indem wir `...T` hinzufügen.

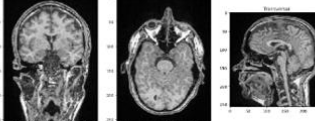
```
[2]: sagittal = image_stack[:, :, 128].T
coronal = image_stack[:, 128, :].T
transverse = image_stack[96]

fig, axs = plt.subplots(1, 3, figsize=(15, 15))

# orthogonale Ebenen anzeigen
axs[0].imshow(sagittal, cmap='Greys_r')
axs[0].set_title('Sagittal')

axs[1].imshow(coronal, cmap='Greys_r')
axs[1].set_title('Coronal')

axs[2].imshow(transverse, cmap='Greys_r')
axs[2].set_title('Transversal');
```



Videos

Wenn ein Bilddatensatz eine zeitliche Dimension hat, nennen wir ihn ein Video. Die Verarbeitung von Videos funktioniert ähnlich wie die mehrkanaliger Bilder und Bildstapel. Öffnen wir einen Mikroskopie-



Pilas de imágenes multidimensionales

Los datos de imágenes multidimensionales se pueden manejar de manera similar a los datos de imágenes multicanal.

Pilas de imágenes tridimensionales

También hay imágenes con tres dimensiones espaciales: X, Y y Z. Puedes encontrar ejemplos típicos en microscopía y en imágenes médicas. Echemos un vistazo a un conjunto de datos de imágenes por Resonancia Magnética (IRM):

```
[1]: from skimage.io import imread
from stackview import imshow
import matplotlib.pyplot as plt
image_stack = imread('.../data/Haase_MRT_tf13d1.tif')
```

Corte de imágenes

Como las tres dimensiones son dimensiones espaciales, también podemos hacer cortes ortogonales al plano de la imagen y correspondientes a Planos Anatómicos. Para orientar correctamente las imágenes, podemos transponer sus ejes añadiendo `...T` al final.

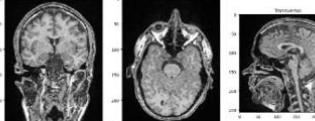
```
[2]: sagittal = image_stack[:, :, 128].T
coronal = image_stack[:, 128, :].T
transverse = image_stack[96]

fig, axs = plt.subplots(1, 3, figsize=(15, 15))

# mostrar planos ortogonales
axs[0].imshow(sagittal, cmap='Greys_r')
axs[0].set_title('Sagital')

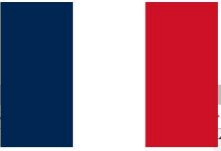
axs[1].imshow(coronal, cmap='Greys_r')
axs[1].set_title('Coronal')

axs[2].imshow(transverse, cmap='Greys_r')
axs[2].set_title('Transversal');
```



Videos

Si un conjunto de datos de imágenes tiene una dimensión temporal, lo llamamos video. Procesar videos funciona de manera similar a las imágenes multicanal u a las pilas de imágenes. Vamos a abrir un



Piles d'images multidimensionnelles

Les données d'images multidimensionnelles peuvent être traitées de manière similaire aux données d'images multi-canaux.

Piles d'images tridimensionnelles

Il existe aussi des images avec trois dimensions spatiales: X, Y et Z. Vous trouvez des exemples typiques en microscopie et en imagerie médicale. Regardons un ensemble de données d'imagerie par résonance magnétique (IRM):

```
[1]: from skimage.io import imread
from stackview import imshow
import matplotlib.pyplot as plt
image_stack = imread('.../data/Haase_MRT_tf13d1.tif')
```

Coupe d'image

Comme les trois dimensions sont des dimensions spatiales, nous pouvons également faire des coupes orthogonales au plan de l'image et correspondant aux plans anatomiques. Pour orienter correctement les images, nous pouvons transposer leurs axes en ajoutant `...T` à la fin.

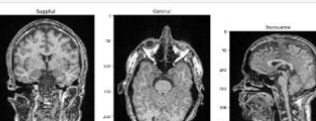
```
[2]: sagittal = image_stack[:, :, 128].T
coronal = image_stack[:, 128, :].T
transverse = image_stack[96]

fig, axs = plt.subplots(1, 3, figsize=(15, 15))

# montrer des plans orthogonaux
axs[0].imshow(sagittal, cmap='Greys_r')
axs[0].set_title('Sagittal')

axs[1].imshow(coronal, cmap='Greys_r')
axs[1].set_title('Coronal')

axs[2].imshow(transverse, cmap='Greys_r')
axs[2].set_title('Transverse');
```

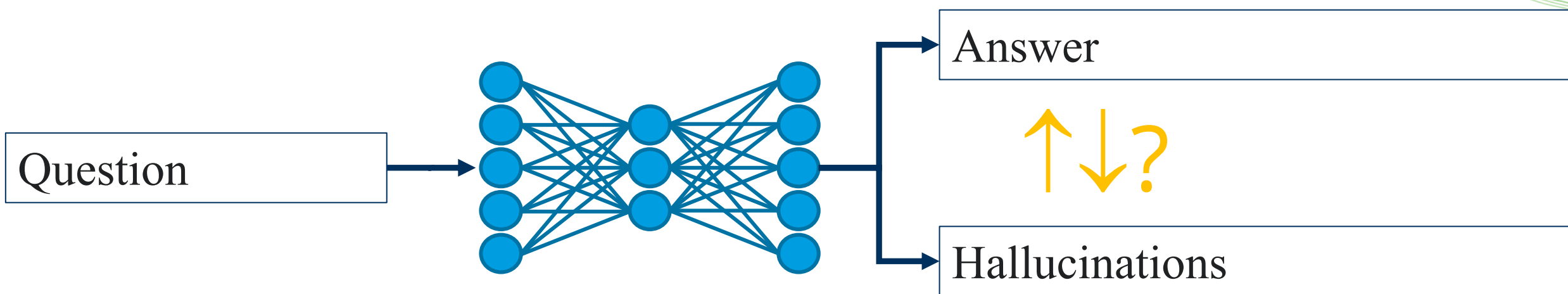


Vidéos

Si un ensemble de données d'image a une dimension temporelle, nous l'appelons une vidéo. Le traitement des vidéos fonctionne de

Large Language Models (LLMs)

Text-to-text, translation, knowledge retrieval



Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

Patrick Lewis^{†‡}, Ethan Perez^{*},

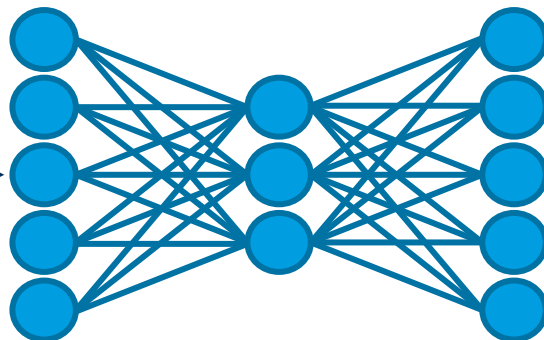
Aleksandra Piktus[†], Fabio Petroni[†], Vladimir Karpukhin[†], Naman Goyal[†], Heinrich Küttler[†],

Mike Lewis[†], Wen-tau Yih[†], Tim Rocktäschel^{†‡}, Sebastian Riedel^{†‡}, Douwe Kiela[†]

Large Language Models (LLMs)

Text-to-text, translation, reasoning

Hypothesis



Proof + Explanation

The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery

Chris Lu^{1,2,*}, Cong Lu^{3,4,*}, Robert Tjarko Lange^{1,*}, Jakob Foerster^{2,†}, Jeff Clune^{3,4,5,†} and David Ha^{1,†}

^{*}Equal Contribution, ¹Sakana AI, ²FLAIR, University of Oxford, ³University of British Columbia, ⁴Vector Institute, ⁵Canada CIFAR AI Chair, [†]Equal Advising

2024-8-16

Large Language Models (LLMs)

Text-to-text, translation, reasoning

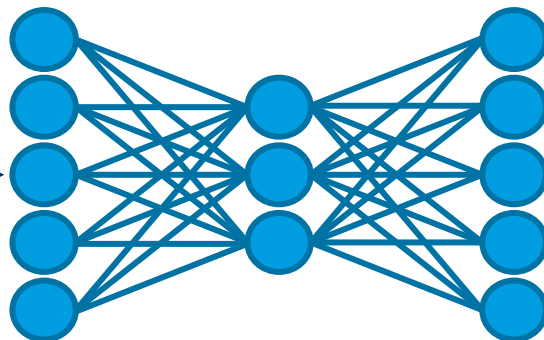


Large Language Models (LLMs)

Text-to-text, translation, code generation

Open hela-cells.tif

English



```
from skimage.io import imread  
image = imread("hela-cells.tif")
```

Python

Published as a conference paper at ICLR 2024

**SWE-BENCH: CAN LANGUAGE MODELS RESOLVE
REAL-WORLD GITHUB ISSUES?**

Carlos E. Jimenez^{* 1,2} John Yang^{* 1,2} Alexander Wettig^{1,2}
Shunyu Yao^{1,2} Kexin Pei³ Ofir Press^{1,2} Karthik Narasimhan^{1,2}

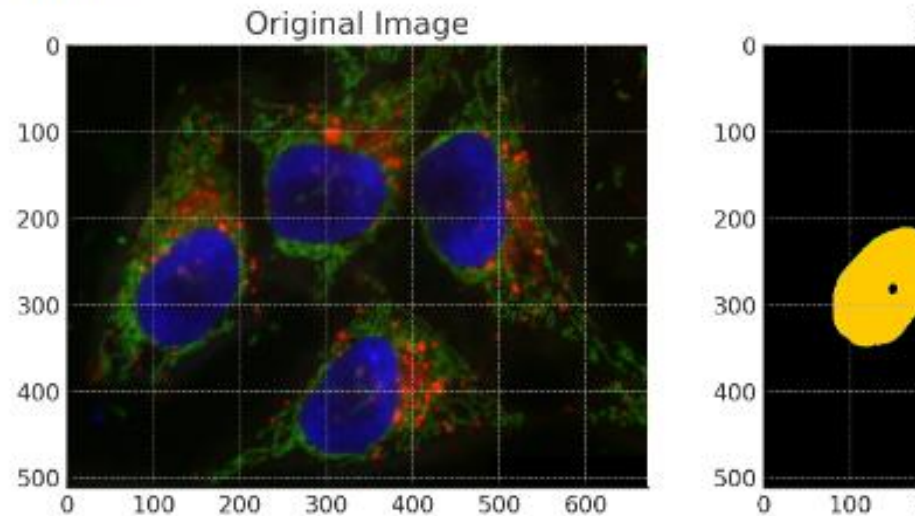
¹Princeton University ²Princeton Language and Intelligence ³University of Chicago

Bio-image Analysis using Large-L

Analysing images in the browser, e.g. using ChatC

Prompt:
“Please segment
the nuclei in the
blue channel of
the image”

ChatGPT



The segmentation of the nuclei has been performed on the blue channel
the figure shows the segmented nuclei in different colors, each corres
the field of view. Each color represents a unique label assigned to the
image. [~]



Analysis

```
from skimage import io, filters, measure, morphology
import matplotlib.pyplot as plt
import numpy as np

# Load the image
image = io.imread('/mnt/data/hela-cells-8bit.png')

# Since it's a color image, we'll just take the blue channel
blue_channel = image[:, :, 2]

# Threshold the image to get the binary image for nuclei
thresh = filters.threshold_otsu(blue_channel)
binary = blue_channel > thresh

# Remove small objects and perform morphological closing
cleaned = morphology.remove_small_objects(binary, 50)
closed = morphology.closing(cleaned, morphology.disk(3))

# Label the image
label_image = measure.label(closed)

# Plot the original image and the binary image
fig, ax = plt.subplots(1, 2, figsize=(12, 6))
ax[0].imshow(image)
ax[0].set_title('Original Image')
ax[1].imshow(label_image, cmap='nipy_spectral')
ax[1].set_title('Segmented Nuclei')
plt.show()
```

Large Language Models (LLMs)

Text-to-text, translation, code generation

Issue

data leak in GBDT due to warm start (This is about the non-histogram-based version of...

Codebase

sklearn/ reqs.txt
examples/ setup.cfg
README.rst setup.py



Language Model



Generated PR

+20 -12

sklearn
gradient_boosting.py
helper.py
utils



Unit Tests

Pre PR	Post PR	Tests
✗	✓	join_struct_col
✗	✓	vstack_struct_col
✗	✓	dstack_struct_col
✓	✓	matrix_transform
✓	✓	euclidean_diff

Published as a conference paper at ICLR 2024

SWE-BENCH: CAN LANGUAGE MODELS RESOLVE REAL-WORLD GITHUB ISSUES?

Carlos E. Jimenez^{*1,2} John Yang^{*1,2} Alexander Wettig^{1,2}
Shunyu Yao^{1,2} Kexin Pei³ Ofir Press^{1,2} Karthik Narasimhan^{1,2}

¹Princeton University ²Princeton Language and Intelligence ³University of Chicago

Chatting with LLMs on GitHub/Lab

Goal: Transparency + knowledge exchange

haesleinhuepf opened last week

I'd like to have python code for classifying images. I think using the resnet50 model from torchvision. We can use this as example image:



Before running the classification, make sure all necessary libraries are installed.

git-bob try to do this.

Create sub-issue



1

github-actions last week

Contributor

This message was generated by [git-bob](#) (version: 0.19.1, model: claude-3-5-sonnet-20241022, [log](#)), an experimental AI-based assistant. It can make mistakes and has [limitations](#). Check its messages carefully.

I worked on implementing image classification using ResNet50 from torchvision for the microscope and cat image. I first saved the input image as microscope_cat.jpg, then created a Jupyter notebook (image_classification.ipynb) that implements the classification logic using the ResNet50 model. The notebook loads the image, preprocesses it, and uses the pre-trained model to predict what's in the image. The classification results were saved to a text file and I made sure to specify the required dependencies in requirements.txt for anyone who wants to run this code.

- [classification_results.txt](#) Contains the top predicted categories from the ResNet50 model for the input image
- [image_classification.ipynb](#) Jupyter notebook implementing the image classification using ResNet50 model from torchvision
- [requirements.txt](#) Lists the Python package dependencies needed to run the classification code

Top 5 predictions:
sewing machine: 0.443
toaster: 0.240
iron: 0.132
soap dispenser: 0.047
vacuum: 0.033

Image Classification using ResNet50

First, let's install and import the necessary libraries.

```
In [1]:  
!pip install torch torchvision pillow requests
```

```
In [2]:  
import torch  
from torchvision import models, transforms  
from PIL import Image  
import requests  
from io import BytesIO
```

Load and preprocess the image

```
In [3]:  
# Define image transformations  
preprocess = transforms.Compose([  
    transforms.Resize(256),  
    transforms.CenterCrop(224),  
    transforms.ToTensor(),  
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])  
)  
  
# Load image  
image = Image.open('microscope_cat.jpg')  
input_tensor = preprocess(image)  
input_batch = input_tensor.unsqueeze(0)
```

Load the pre-trained ResNet50 model and run inference

```
In [4]:  
# Load model and weights  
weights = models.ResNet50_Weights.IMAGENET1K_V1  
model = models.resnet50(weights=weights)  
model.eval()  
  
# Run inference  
with torch.no_grad():  
    output = model(input_batch)  
  
# Get class names  
categories = weights.meta['categories']  
  
# Get top 5 predictions  
_, indices = torch.sort(output[0], descending=True)  
probabilities = torch.nn.functional.softmax(output[0], dim=0)  
  
# Save results  
with open('classification_results.txt', 'w') as f:  
    f.write('Top 5 predictions:\n')  
    for idx in indices[:5]:  
        f.write(f'{categories[idx]}: {probabilities[idx].item():.3f}\n')
```

Why **code generation** using LLMs?

Output of LLMs is often criticized for

- Limited reproducibility
- Hard to evaluate (automatically)

Output of LLM-generated code is

- Reproducible
- Can be executed and evaluated (automatically)

Say a

Say a

Write Python code that says "a".

A.

< 2/2 >

A!

Certainly! Here's a simple Python code snippet that prints the letter "a":

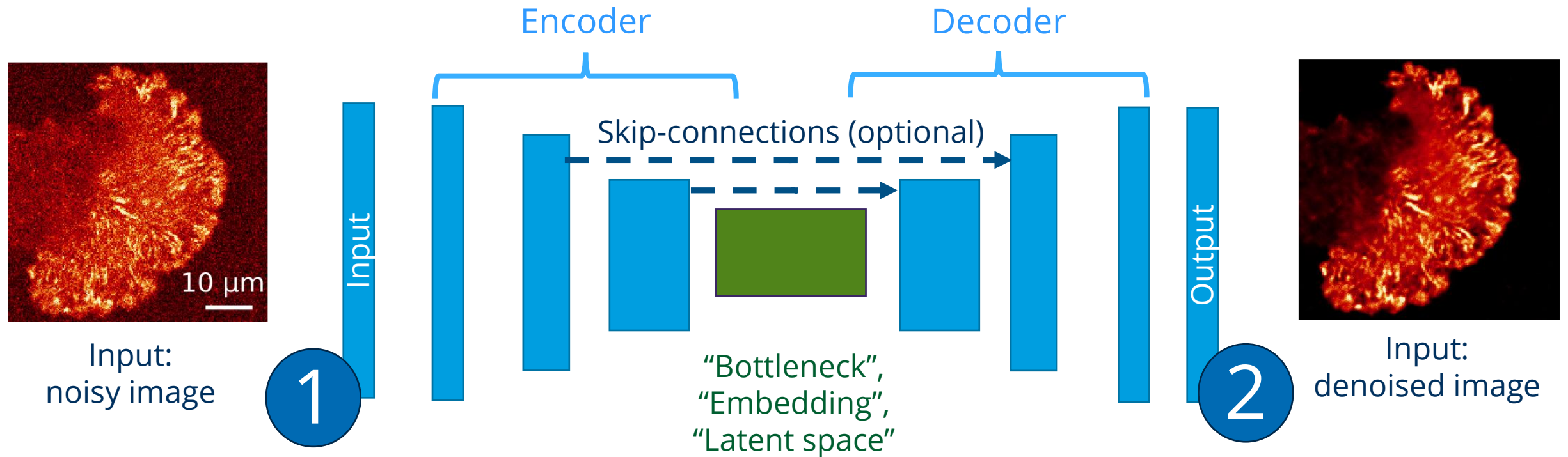
```
python  
  
print("a")
```

No matter how often you execute this code, the output is always guaranteed to be „a“.

Transformer neural networks

Recap: Encoder-Decoder Networks

Related: „Auto-encoder“, „Variational Auto-Encoder“, „U-Net“

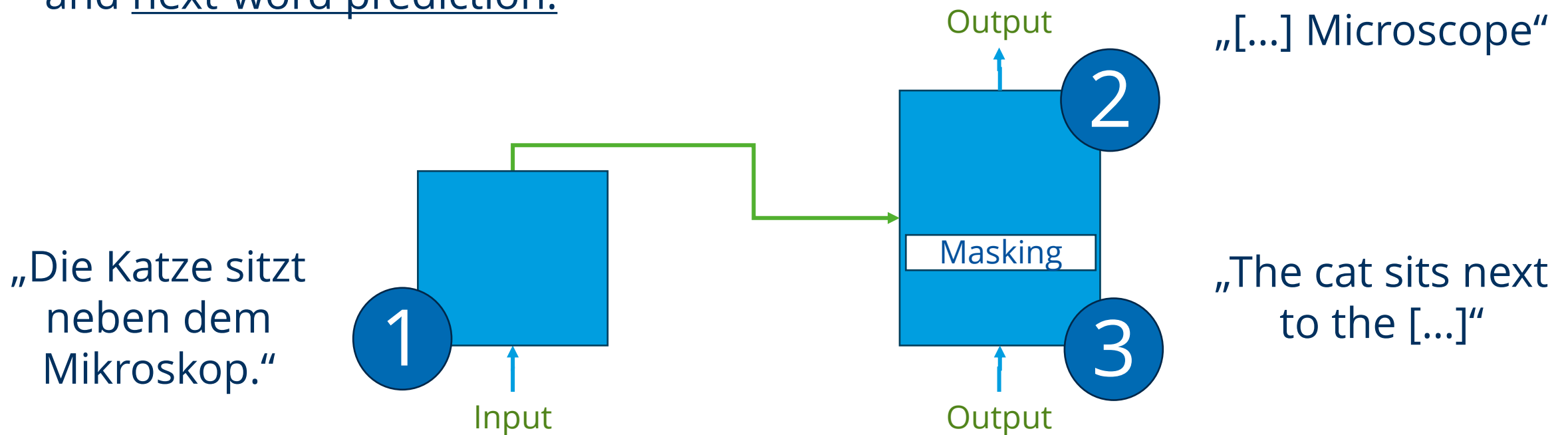


Transformers

LLMs use the **transformer** neural network architecture

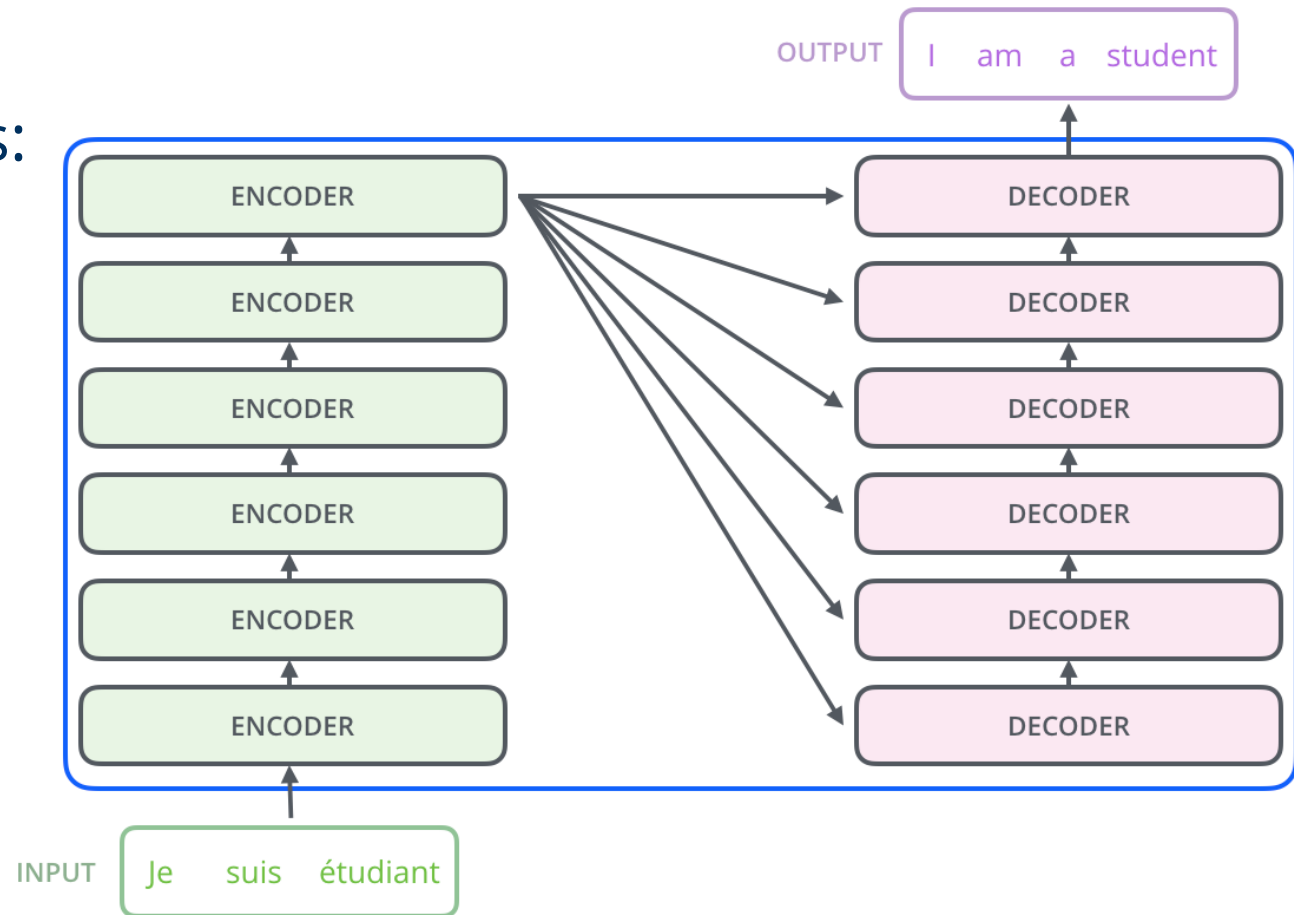
Related: Generative Pretrained **Transformer** (GPT)

LLMs were originally developed for translation tasks and next-word prediction:

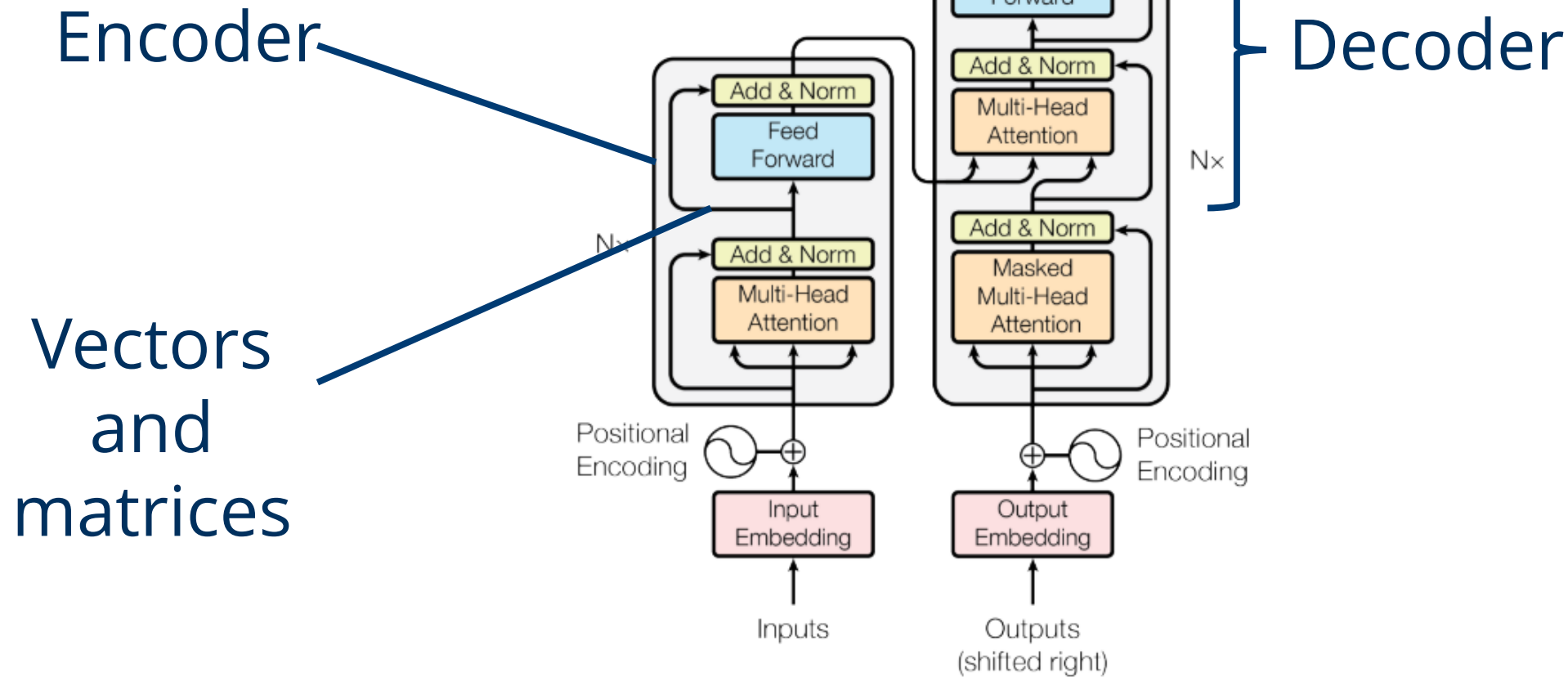


Generative Pretrained Transformer (GPT)

Stacks of encoders and decoders arranged like this:



Generative Pretrained Transformer (GPT)

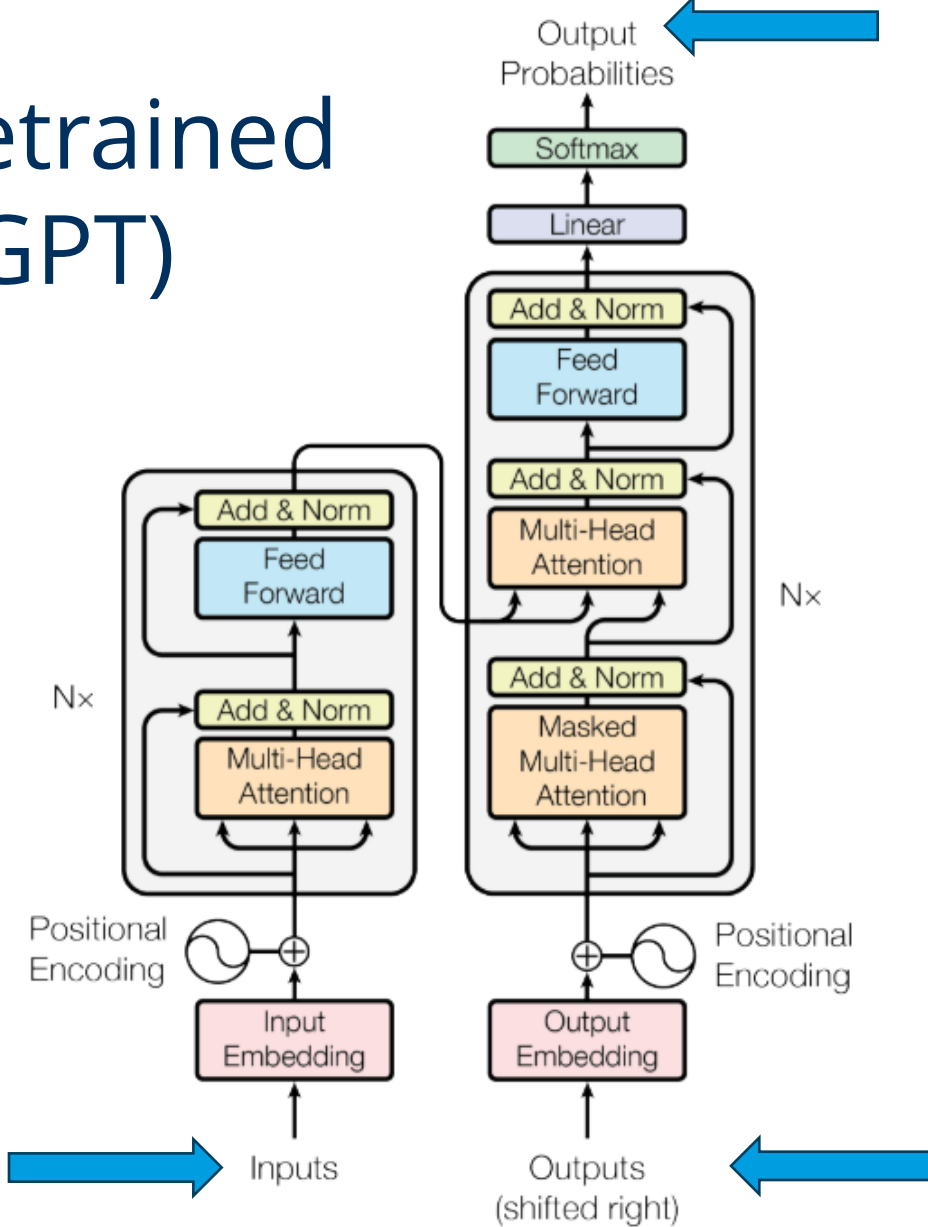


Generative Pretrained Transformer (GPT)

Task: Translation

Example input:

Die Katze sitzt neben dem *Mikroskop*



Next word probabilities
(according to context):
The: 0.9
A: 0.1

Example output:

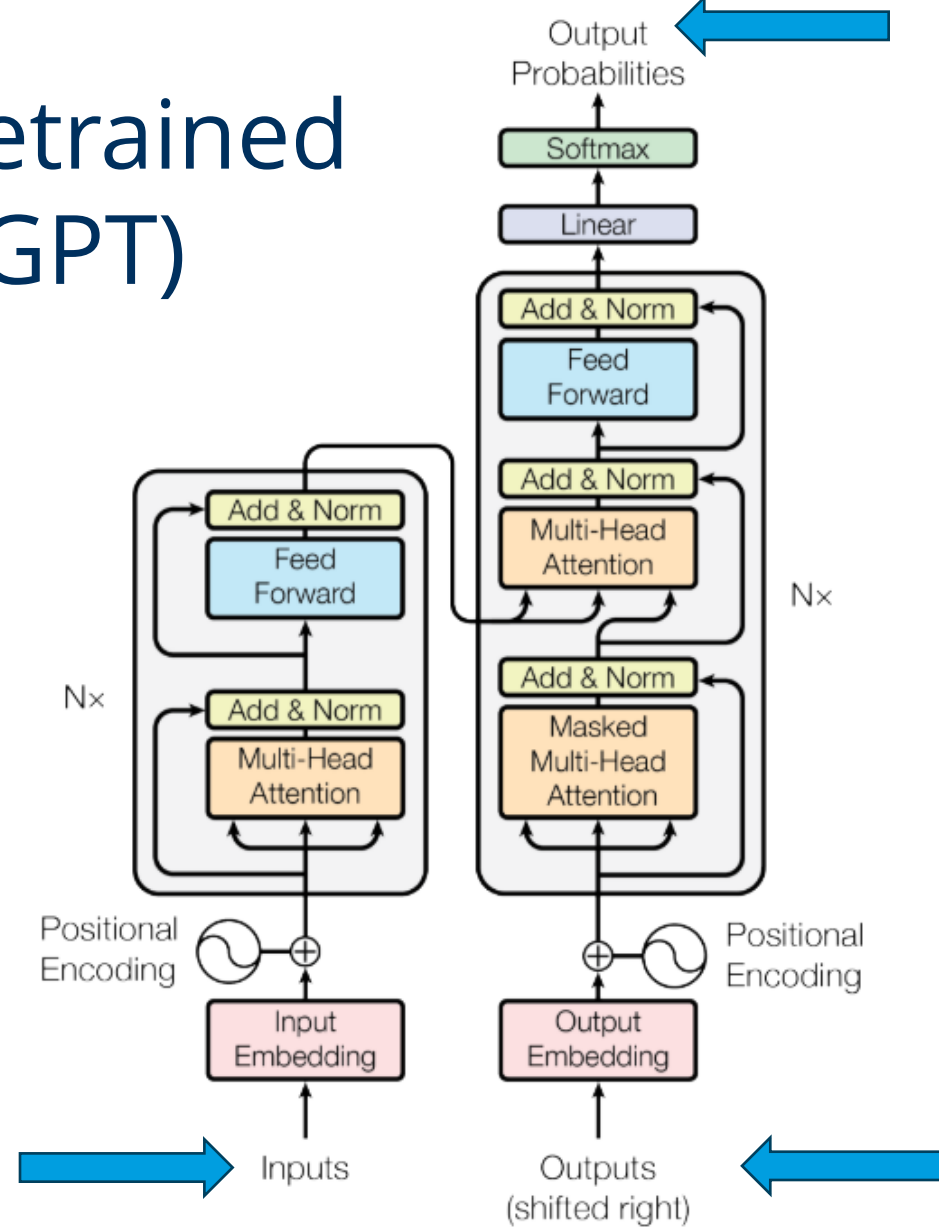
The cat sits next to the microscope

Generative Pretrained Transformer (GPT)

Task: Translation

Example input:

Die Katze sitzt neben dem *Mikroskop*



Next word probabilities
(according to context):
cat: 0.9
dog: 0.0001

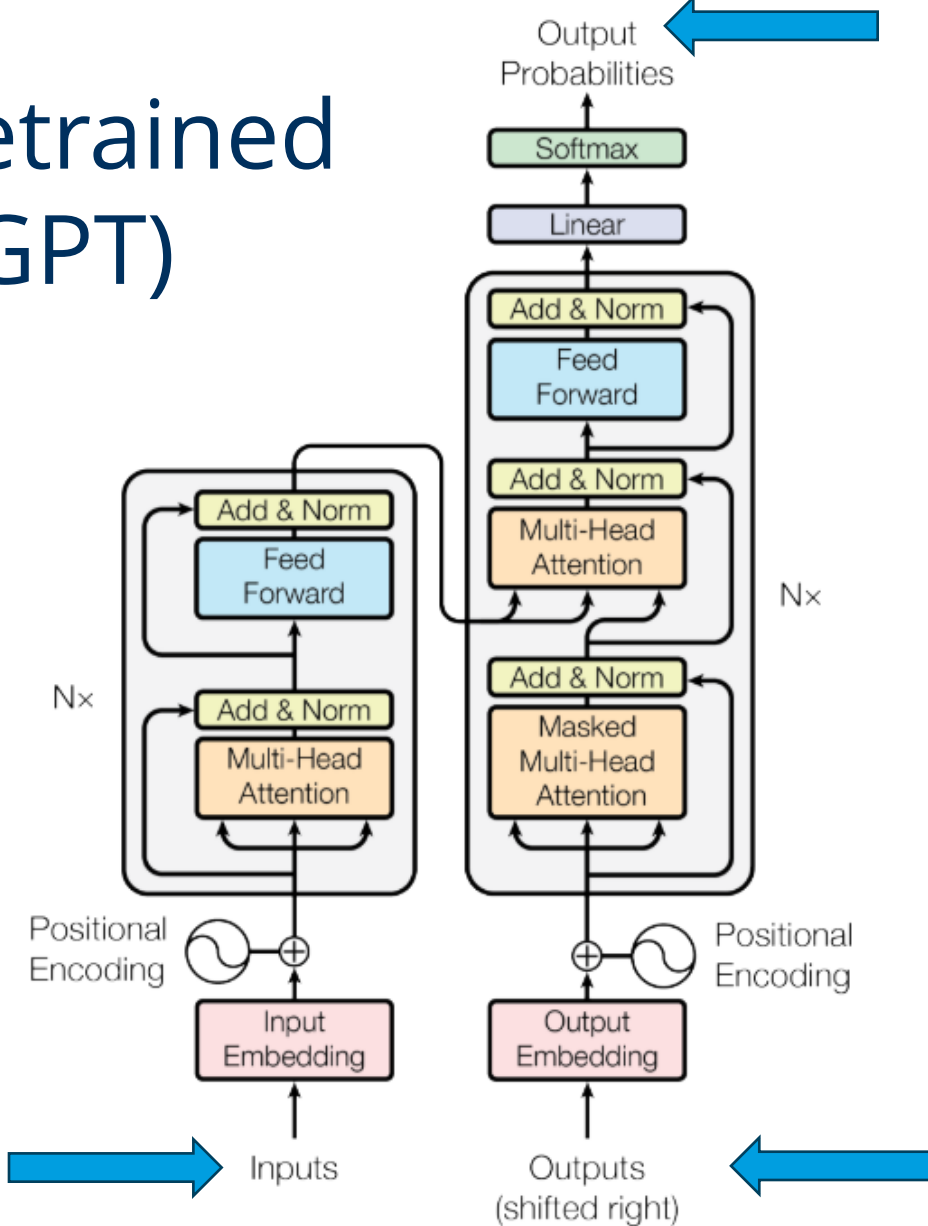
Example output:

The cat sits next to the microscope

Generative Pretrained Transformer (GPT)

Task: Translation

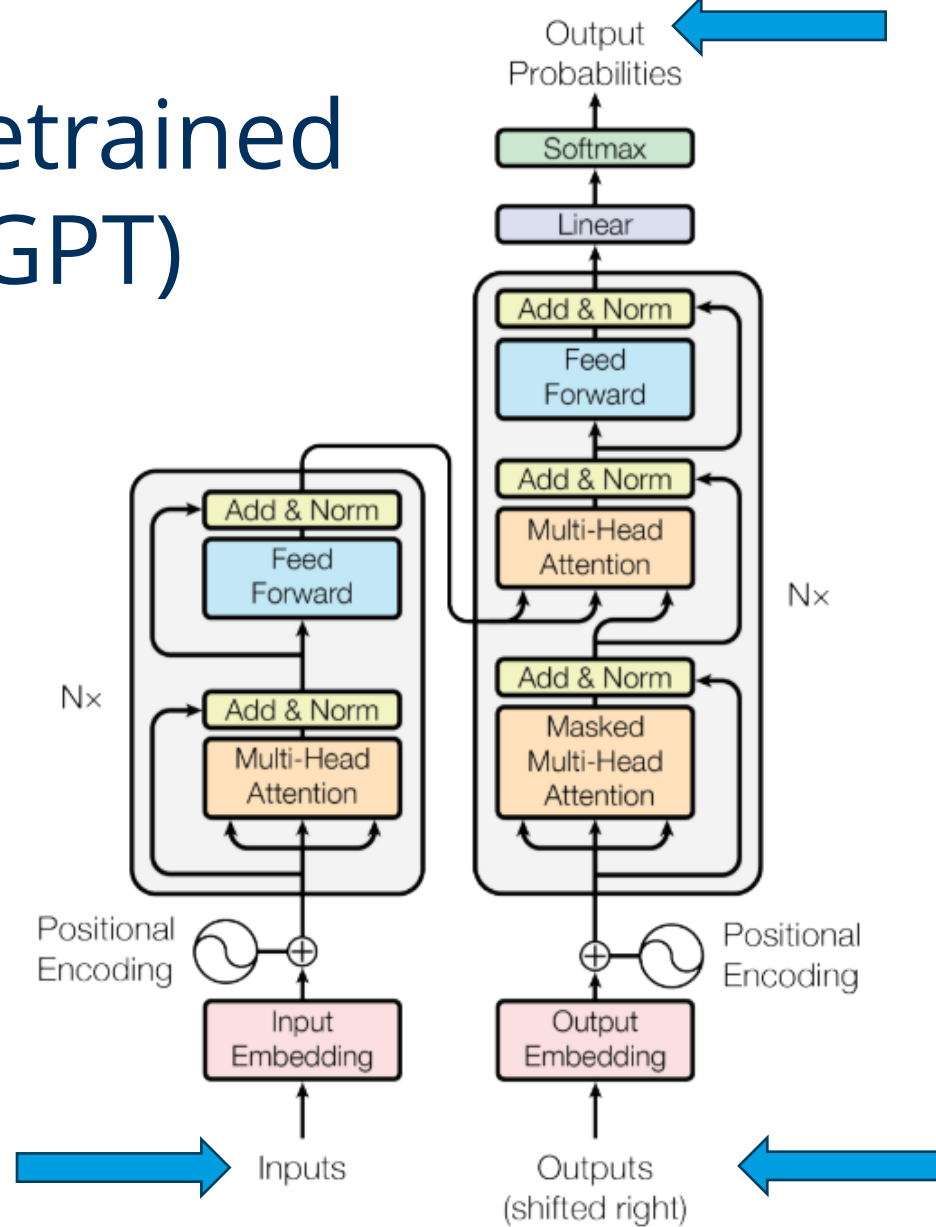
Example input:
Die Katze sitzt neben dem *Mikroskop*



Next word probabilities
(according to context):
sits: 0.8
takes a seat: 0.2

Generative Pretrained Transformer (GPT)

Task: Translation



Next word probabilities
(according to context):
next to: 0.6
near: 0.4

Example input:

Die Katze sitzt neben dem *Mikroskop*

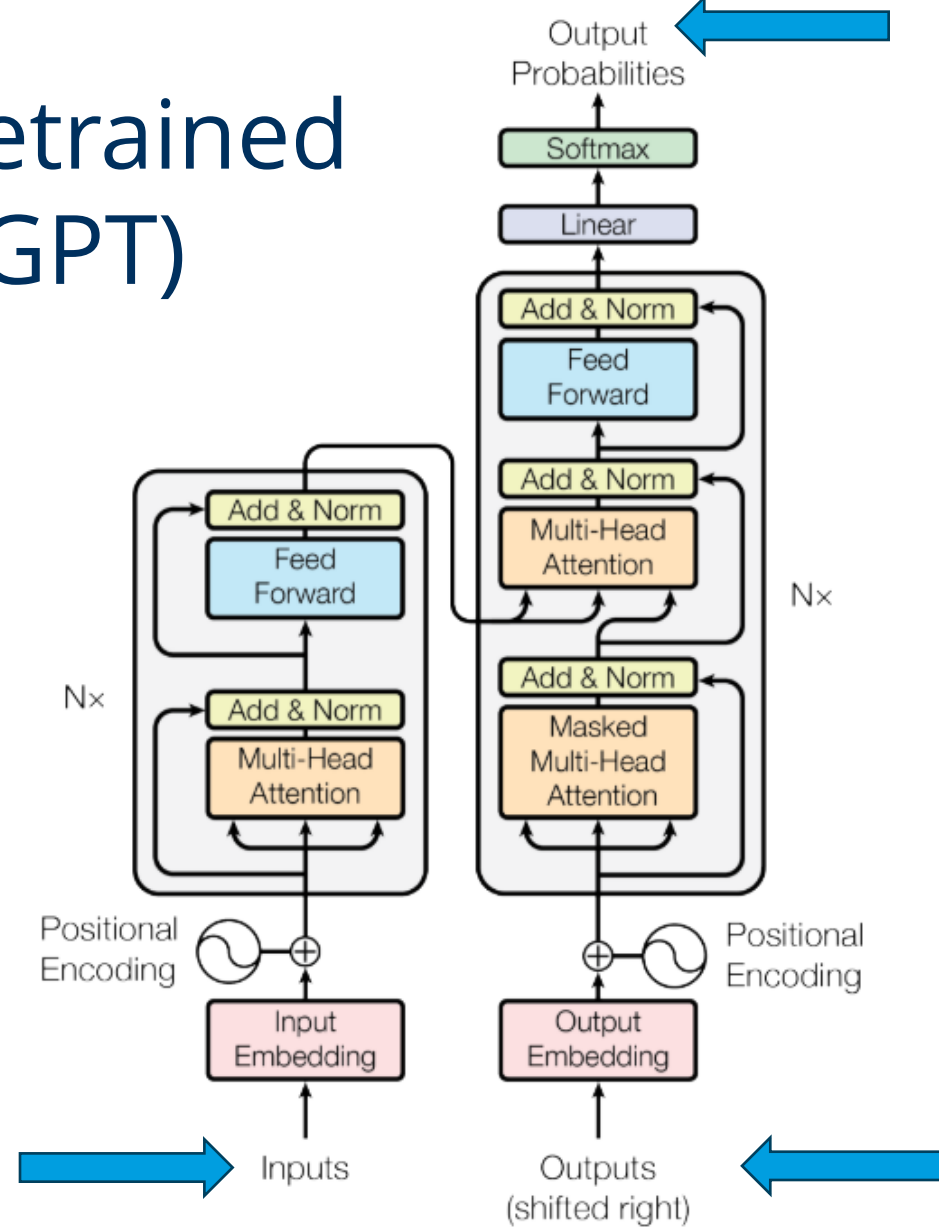
Example output:

The cat sits next to the microscope

Generative Pretrained Transformer (GPT)

Task: Translation

Example input:
Die Katze sitzt neben dem *Mikroskop*



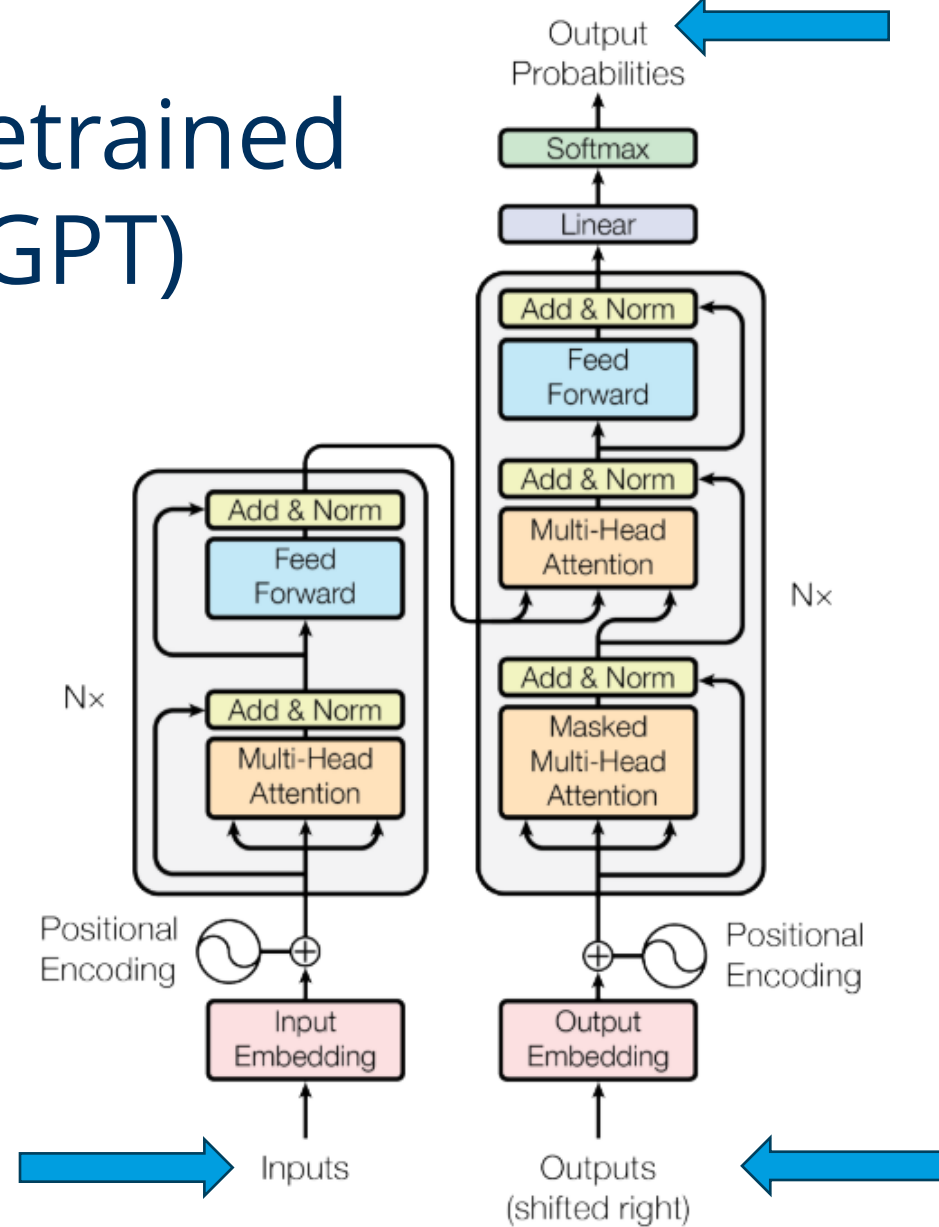
Next word probabilities
(according to context):
the: 0.9
a: 0.1

Example output:
The cat sits next to the microscope

Generative Pretrained Transformer (GPT)

Task: Translation

Example input:
Die Katze sitzt neben dem *Mikroskop*

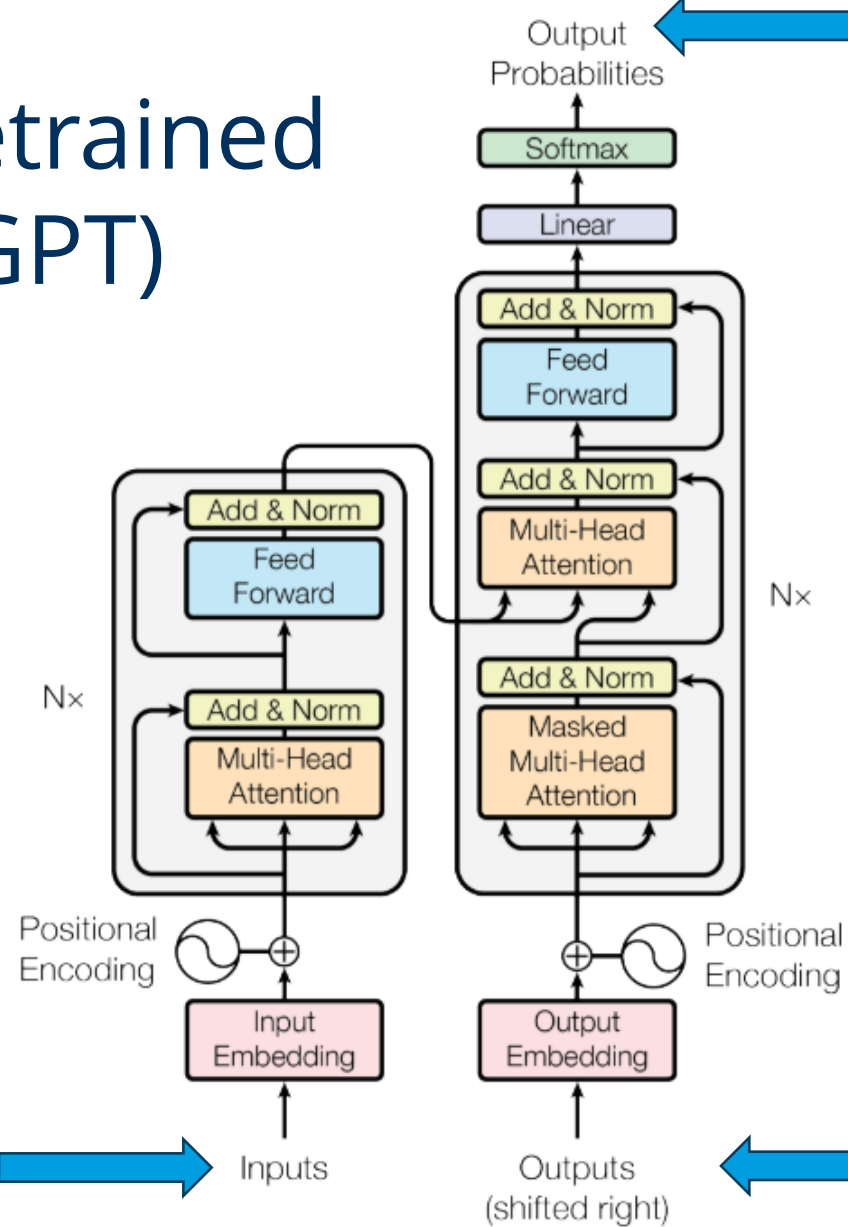


Next word probabilities
(according to context):
Heating: 0.9
Food: 0.8
Dog: 0.2
Microscope: 0.01

Example output:
The cat sits next to the microscope

Generative Pretrained Transformer (GPT)

Task: Translation



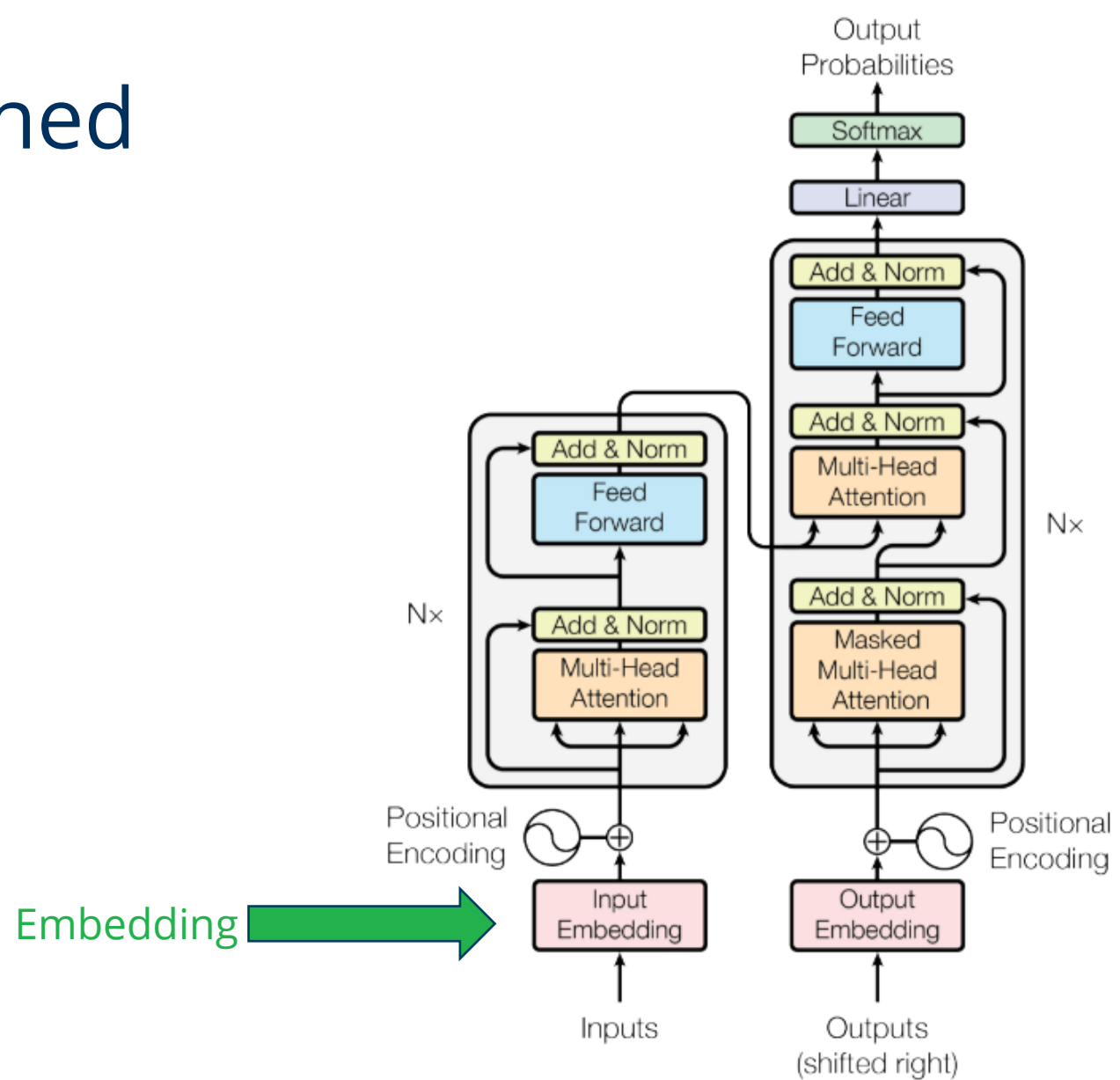
Next word probabilities
(according to context):
Heating: 0.9
Food: 0.8
Dog: 0.2
Microscope: 0.2

Through fine-tuning the transformer, you can increase the probabilities of words.

Example input:
Die Katze sitzt neben dem *Mikroskop*

Example output:
The cat sits next to the microscope

Generative Pretrained Transformer (GPT)

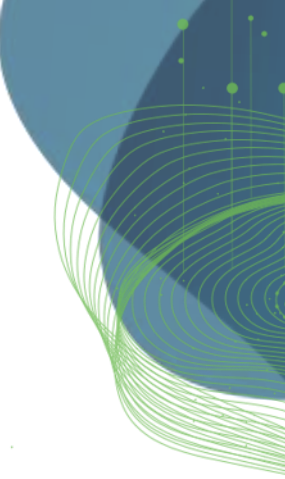




Quiz

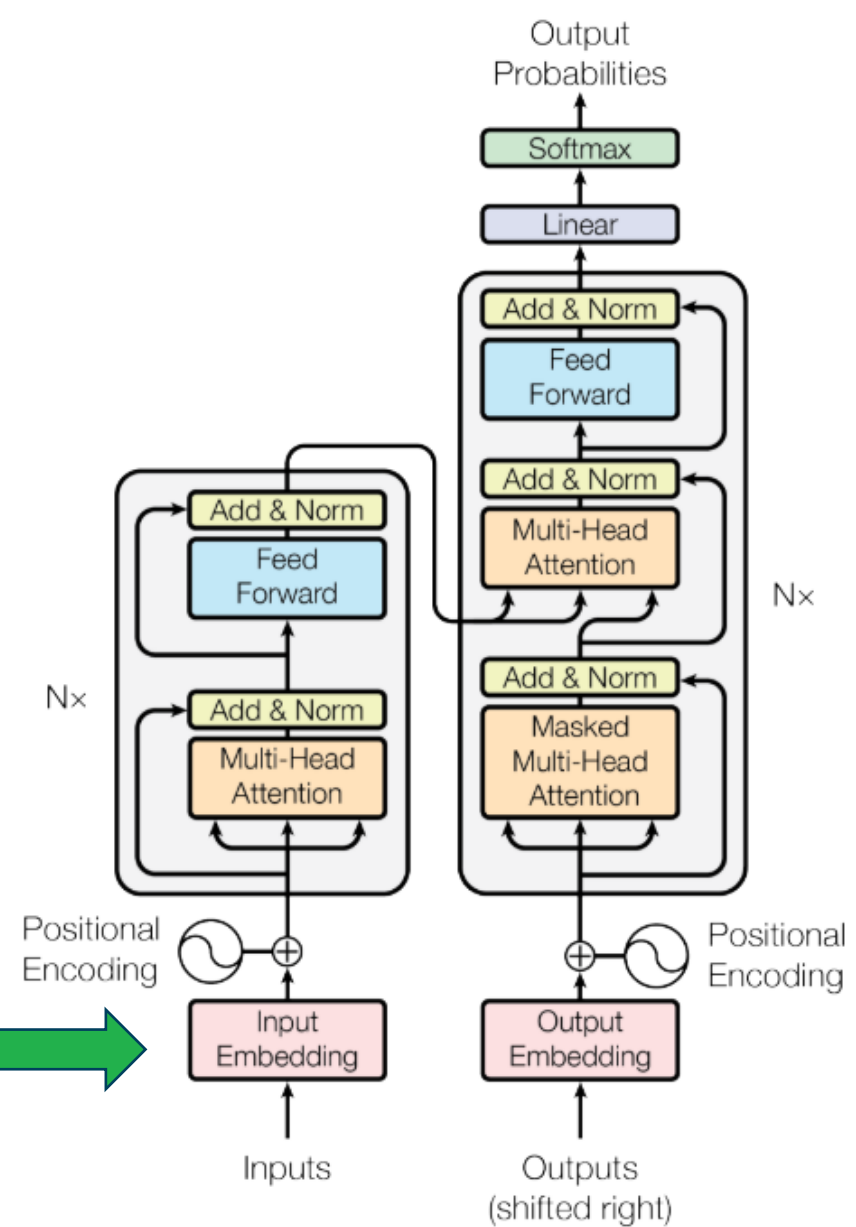
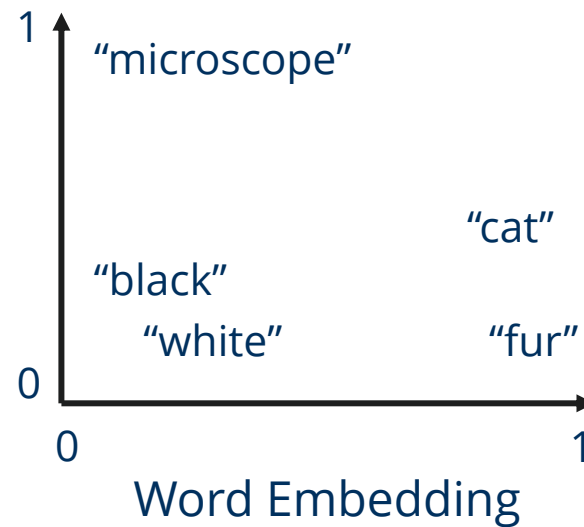
What is an **embedding**?

- a) in the context of image processing
- b) in the context of neural networks
- c) in general



Generative Pretrained Transformer (GPT)

Words need to be converted into vectors to enable NNs to process them.

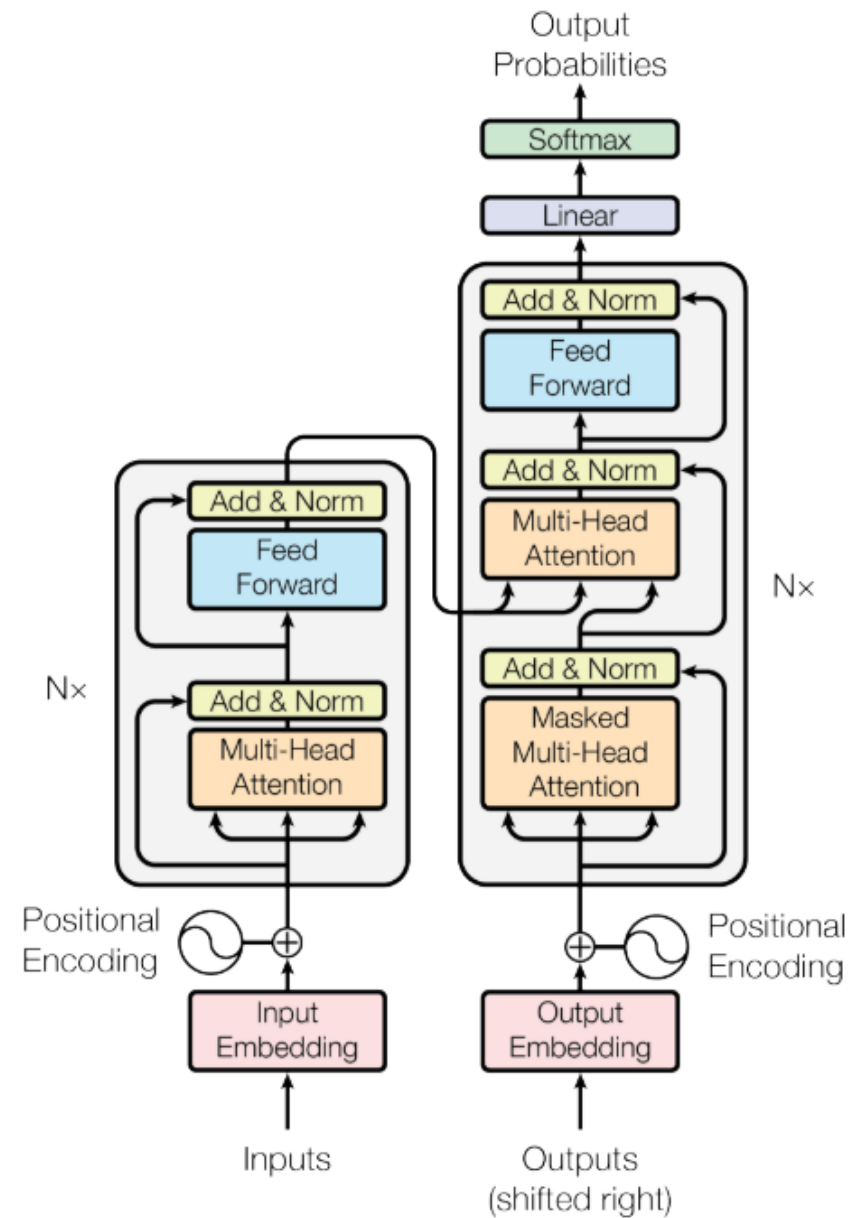


Attention is all you need

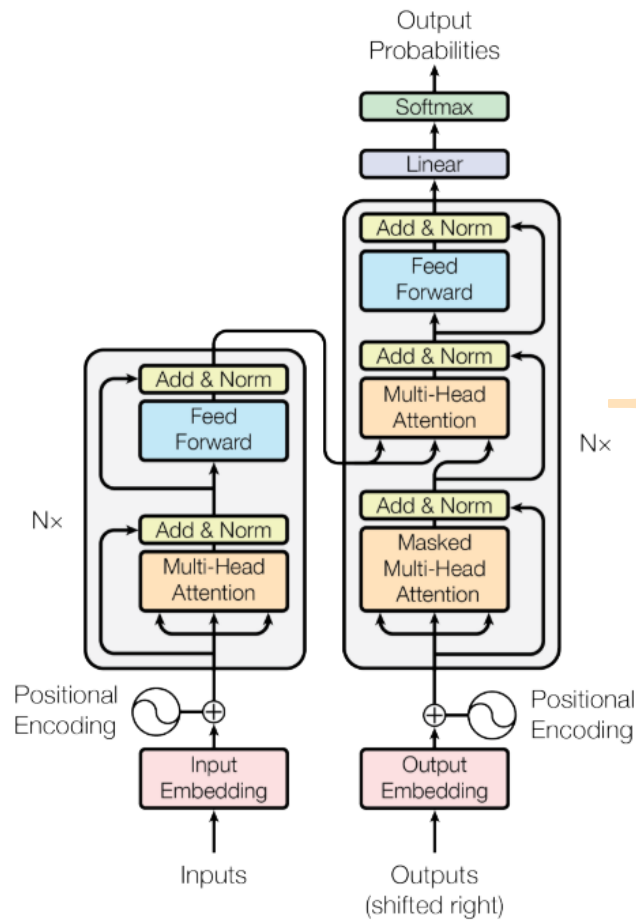
The position of the word in the sentence / context may have influence on its meaning. The position is encoded in a vector as well.

The cat sits next to a **microscope**.

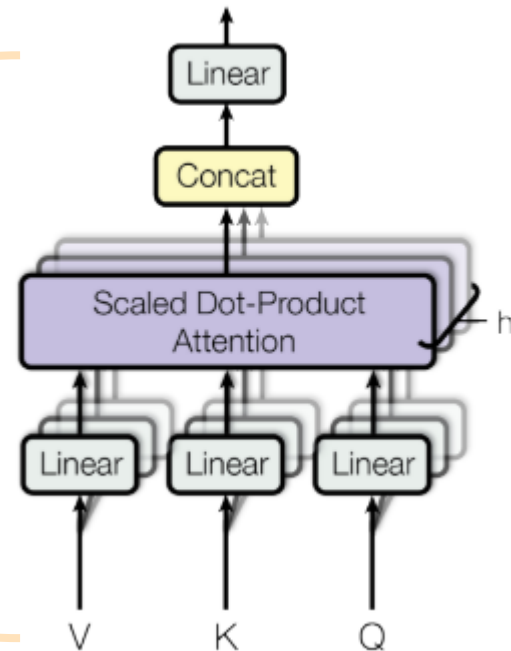
Next to the **microscope** there is a cat.



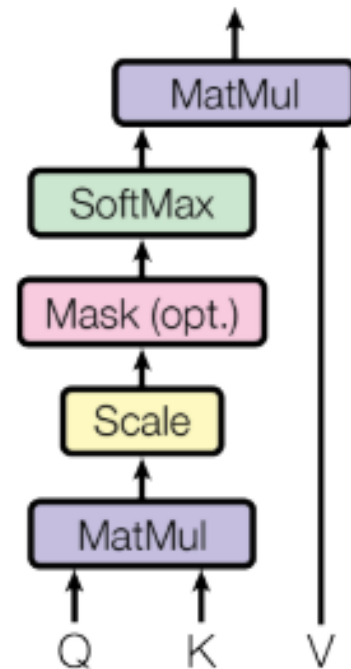
Attention is all you need



Multi-Head Attention



Scaled Dot-Product Attention



Scaled dot-product attention

Attention score: How much related are two words?

Query: For which word are we calculating attention?

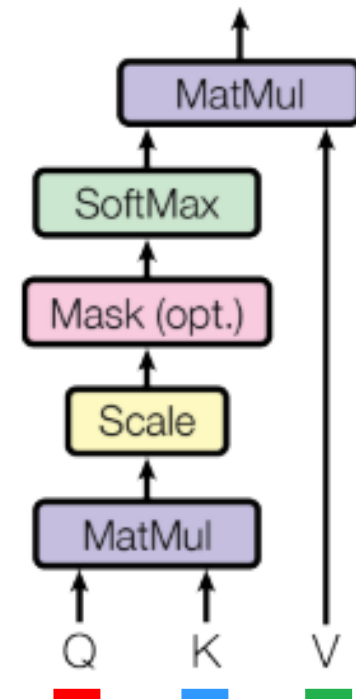
Key: To which word are we calculating attention

Value: Relevance of the query-key relationship

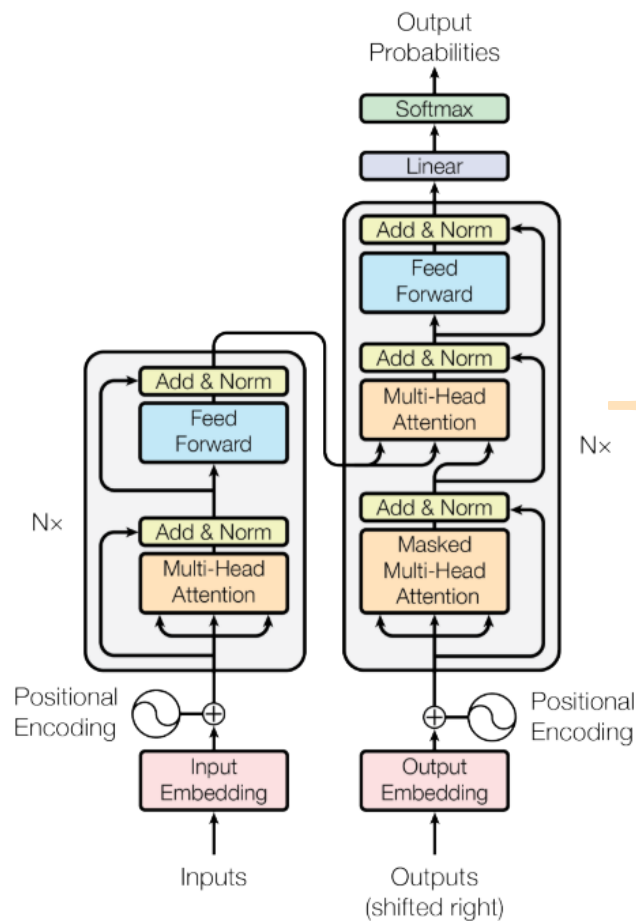
The **cat** is black and **white**.
Relevance value: 0.1

The **cat** is **meowing**.
Relevance value: 0.9

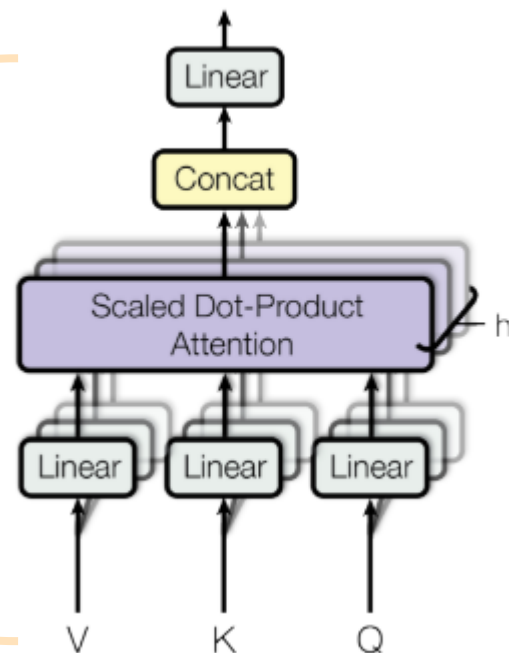
Scaled Dot-Product Attention



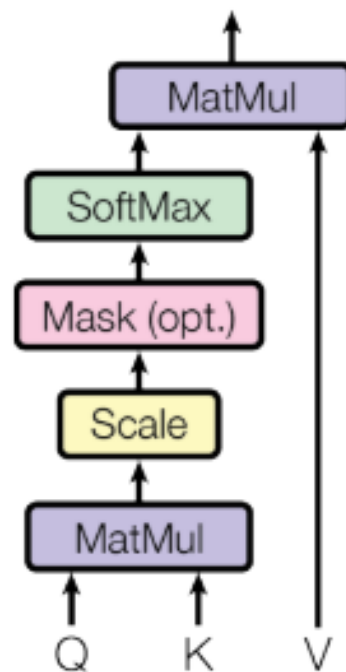
Attention is all you need



Multi-Head Attention

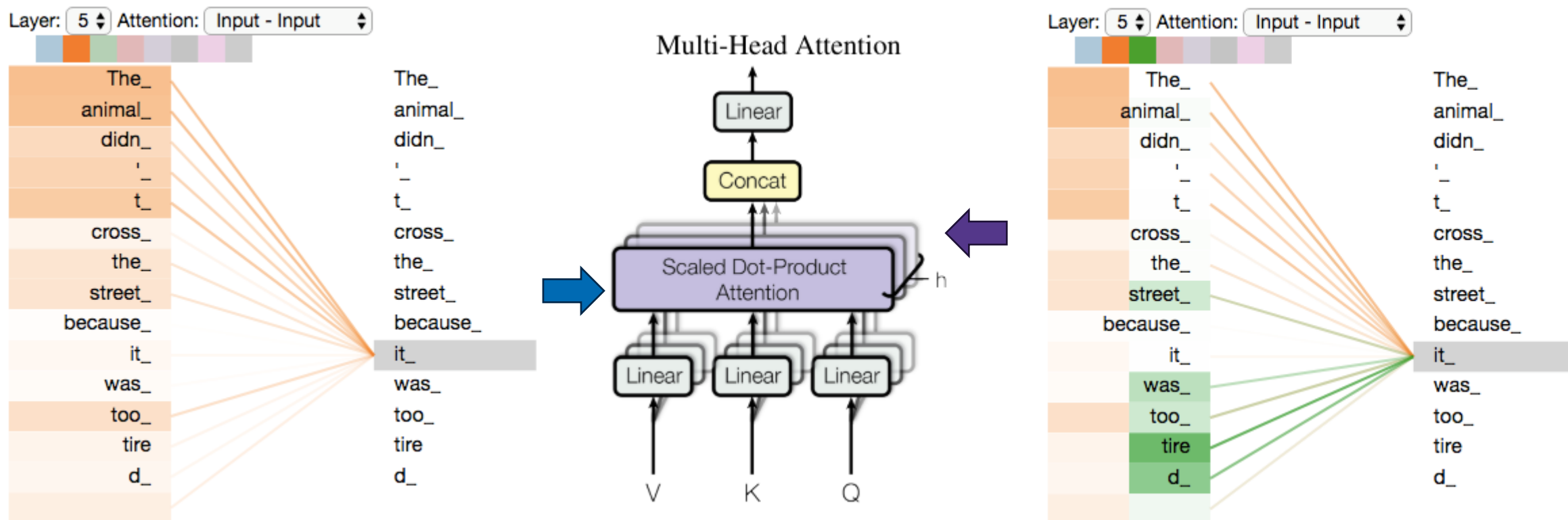


Scaled Dot-Product Attention

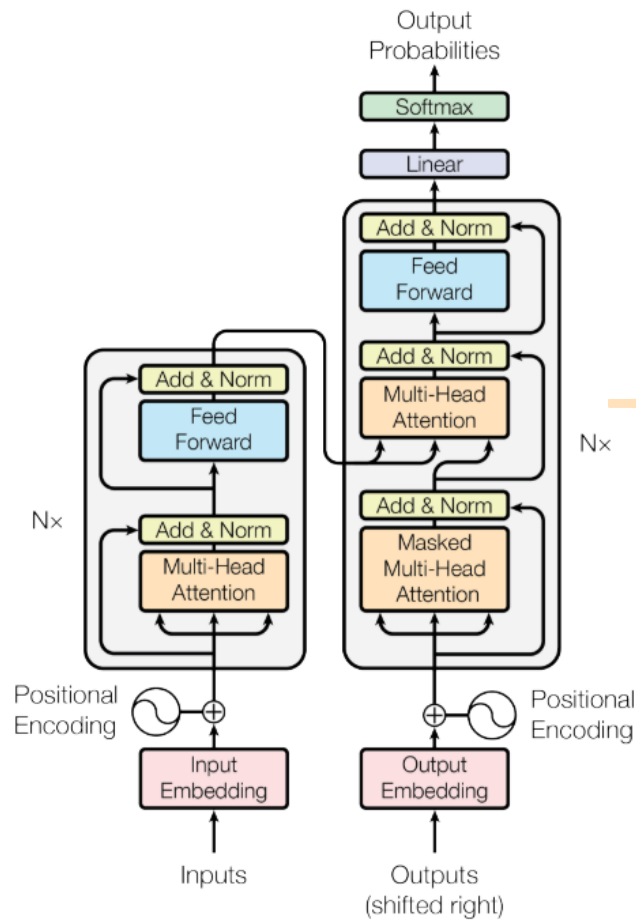


Multi-head attentions

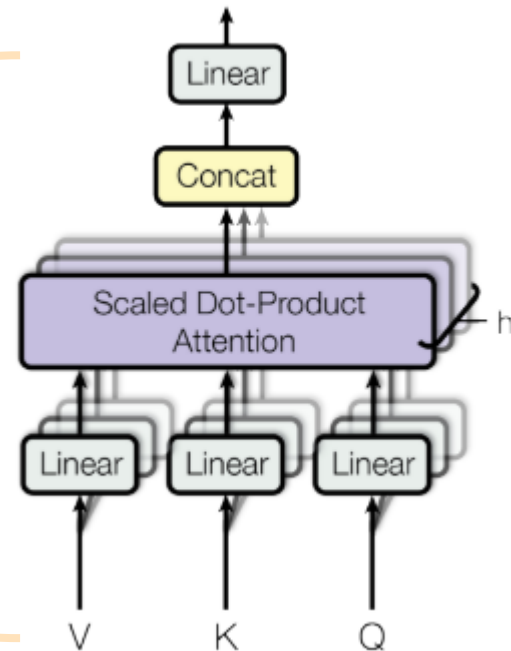
Multiple aspects represented by multiple attention heads



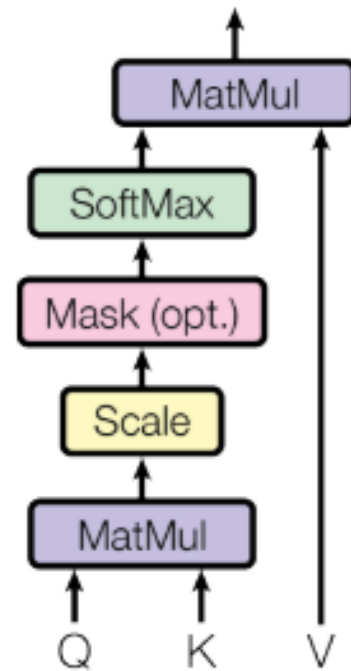
Attention is all you need



Multi-Head Attention

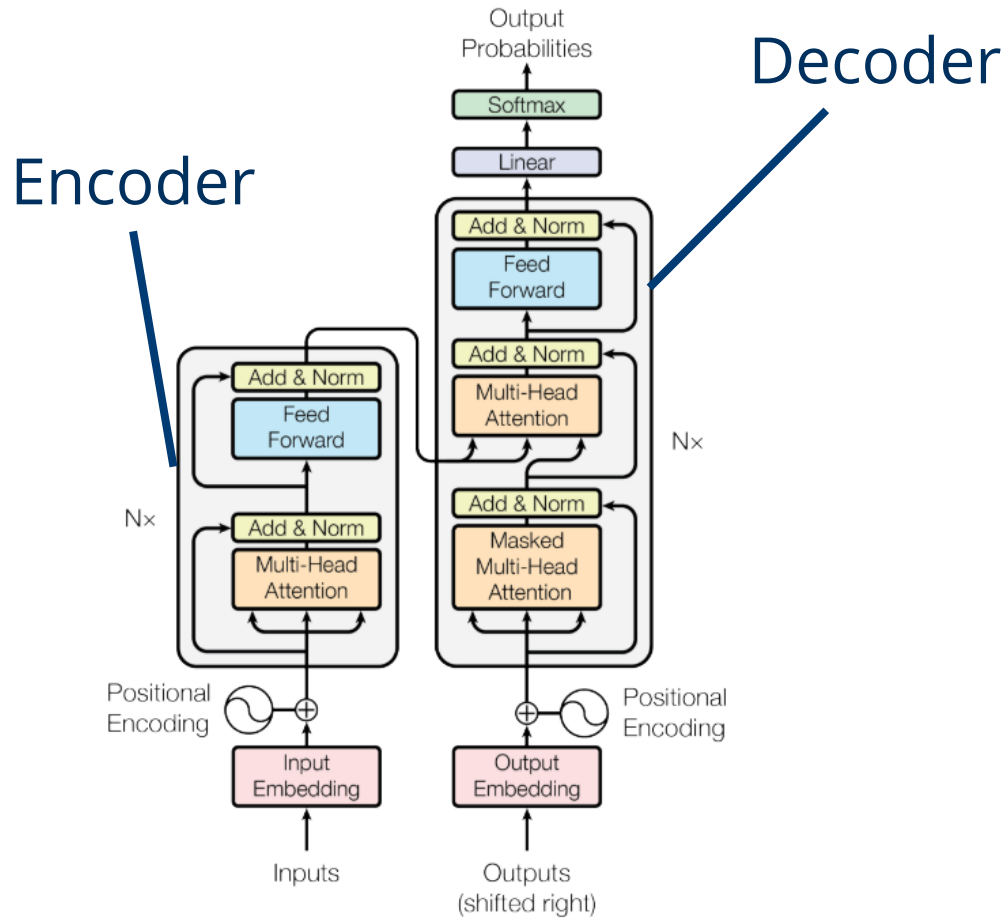


Scaled Dot-Product Attention

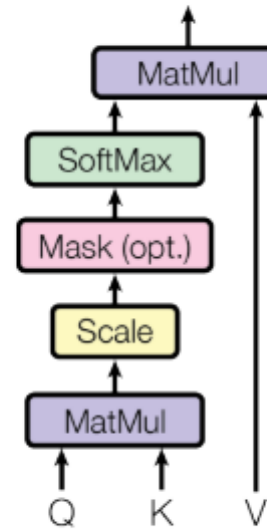


Attention is all you need

Summary



Scaled Dot-Product Attention



The **cat** is black and **white**. Relevance

attention score

The **cat** is **meowing**. Relevance

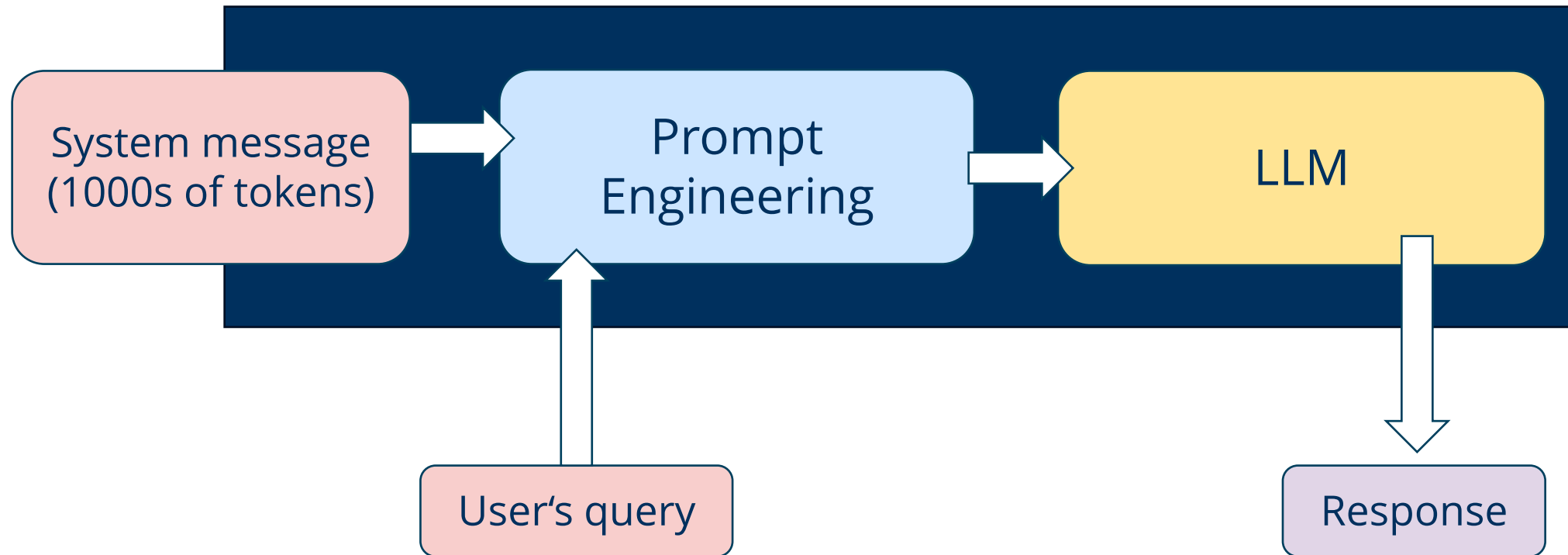
attention score

Prompt Engineering

Long-context prompting

Also known as *in-context learning*

Context with plenty of knowledge provided in system message



Prompt Engineering

Modify a prompt until it works.

- Set context / assign role
- Define output style
- Give hints / conditions
- List tasks
- Provide data

%%prompt

You are a professional academic consultant who gives advice to students **for** writing their thesis.

Give these hints:

- * Submit the thesis registration form only **if** you're **sure** you can finish writing in 5 months.
- * Start writing your thesis early, e.g. **the** fundamentals chapter can be written **from** day one.
- * Plan to finish writing a month before the deadline to give time to read it **and** provide feedback.

Here are some important hints to consider while writing your thesis:

1. Evaluate your readiness: Before submitting the thesis registration form, carefully assess your commitment and availability. Ensure that you are confident in completing your thesis within the given timeframe of five months. This will prevent unnecessary stress and help you maintain a focused mindset throughout the writing process.
2. Begin early: Starting your thesis early is crucial. Begin by working on the fundamentals chapter, as it lays the foundation for your entire thesis. By initiating this chapter from day one, you will establish a good writing routine and make steady progress as you move forward.
3. Plan ahead: Create a realistic schedule to guide your writing process. Break down the work into smaller tasks, setting deadlines for each chapter or section. This will help you maintain a structured approach and make the workload more manageable. Be sure to allocate time for research, data analysis, and revisions.
4. Aim to finish ahead of time: Plan to complete your thesis writing at least one month before the official deadline. This buffer time will allow you to thoroughly review and revise your work before final submission. It will also provide an opportunity to seek feedback from professors or peers, improving the overall quality of your thesis.

Prompt Engineering

Modify a prompt until it works.

- Set context / assign role
- Define output style
- Give hints / conditions
- List tasks
- Provide data

%%prompt

You are a professional academic consultant who gives advice to students **for** writing their thesis.
Keep it short.

Give these hints:

- * Submit the thesis registration form only **if** you're **sure** you can finish writing in 5 months.
- * Start writing your thesis early, e.g. the fundamentals chapter can be written **from** day one.
- * Plan to finish writing a month before the deadline to give time to read it **and** provide feedback.

As a professional academic consultant, here are some helpful hints for writing your thesis:

1. Only submit the thesis registration form if you are confident about completing it within the given timeframe of 5 months.
2. Begin writing your thesis early to ensure a smooth process. For instance, you can start with the fundamentals chapter right from day one.
3. Plan your schedule effectively to finish writing your thesis at least a month before the submission deadline. This will allow ample time for reading, revising, and receiving feedback.

By following these recommendations, you'll have a better chance of successfully completing your thesis on time and with improved quality. Good luck!

Prompt Engineering

Modify a prompt until it works.

- Set context / assign role
- Define output style
- Give hints / conditions
- List tasks
- Provide data

%%prompt

You are a professional academic consultant who gives advice to students **for** writing their thesis.
Keep it short.

Write like Yoda would.

Give these hints:

- * Submit the thesis registration form only **if** you're **sure** you can finish writing in 5 months.
- * Start writing your thesis early, e.g. **the** fundamentals chapter can be written **from** day one.
- * Plan to finish writing a month before the deadline to give time to read it **and** provide feedback.

Hints for thesis writing, I have. Follow them, you must.

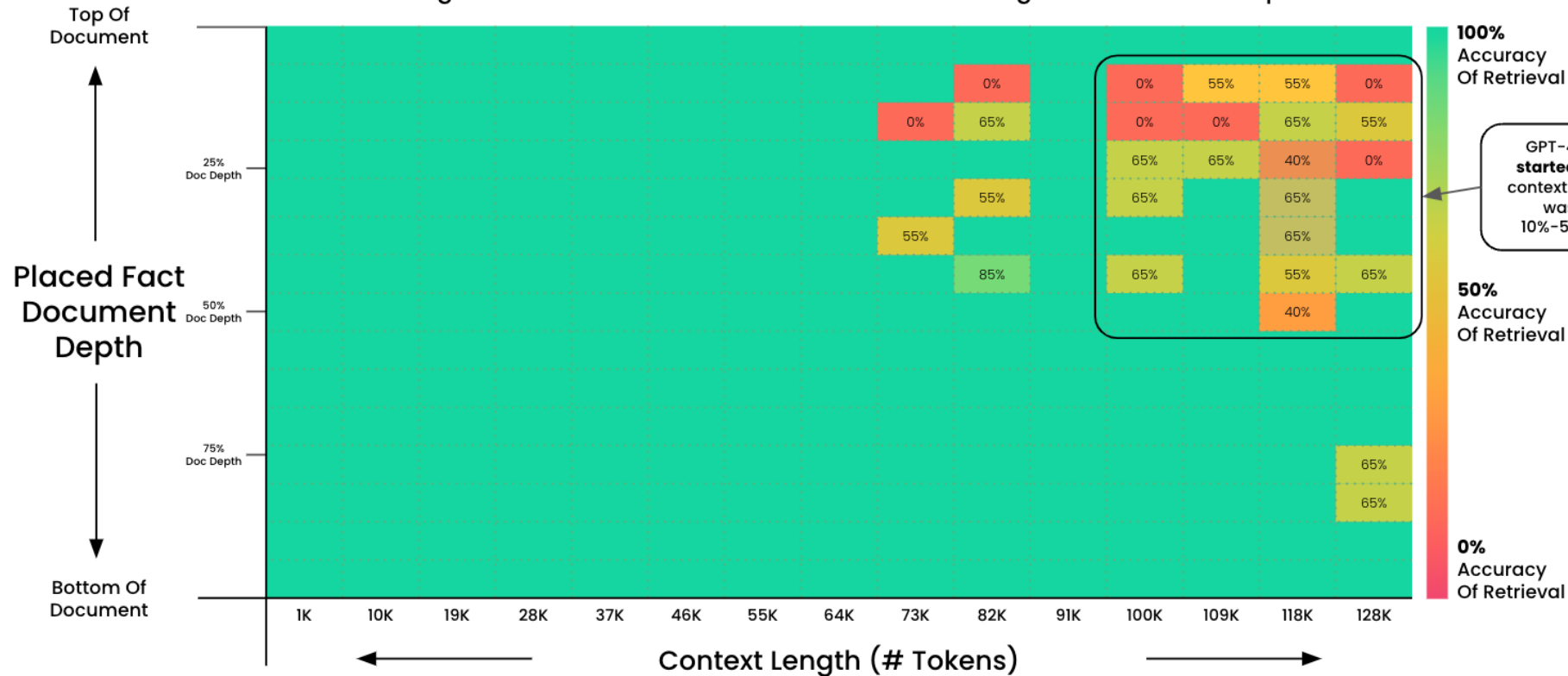
- Only submit the thesis registration form, if in 5 months, complete writing, you are sure.
- From day one, start writing your thesis, you must. Begin with the fundamentals chapter.
- A month before the deadline, finish writing, you should. Time to read and provide feedback, it will give you.

Follow these hints, you must. Successful thesis writing, you will achieve. Hmmm.

Prompt engineering

Pressure Testing GPT-4 128K via "Needle In A HayStack"

Asking GPT-4 To Do Fact Retrieval Across Context Lengths & Document Depth



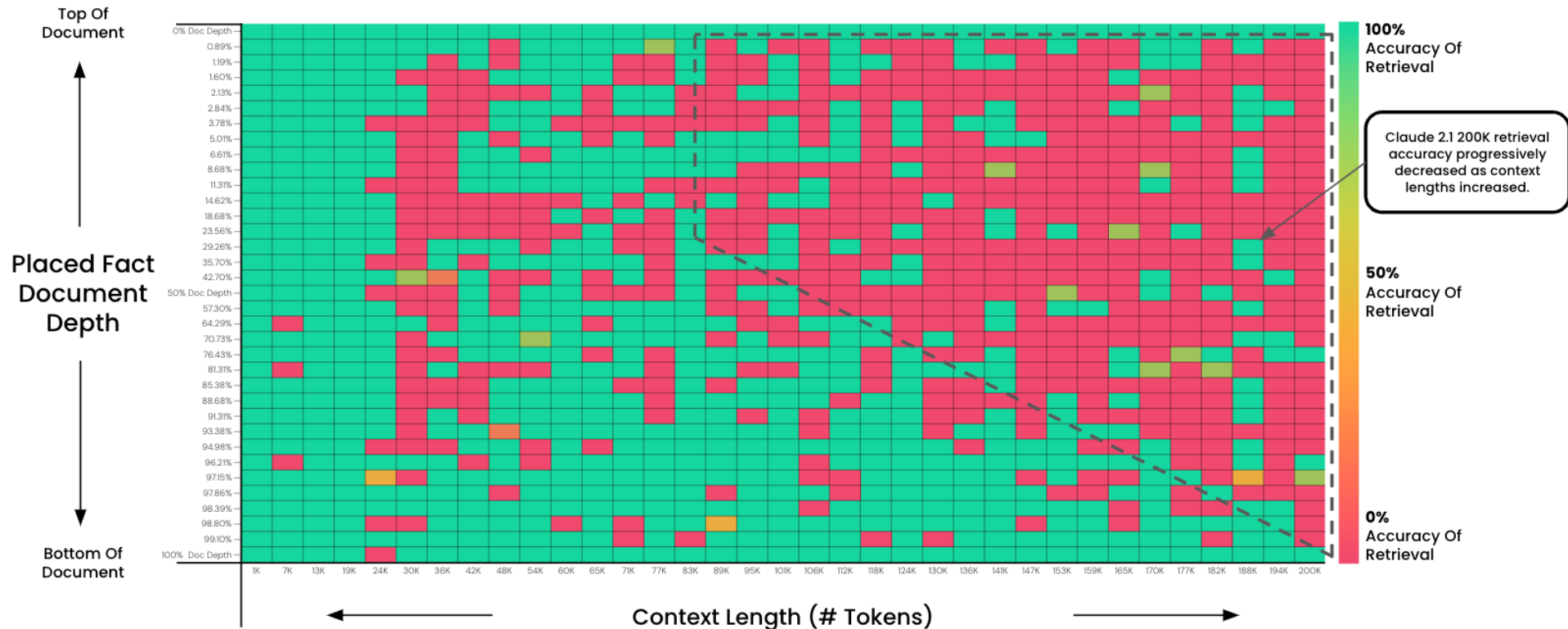
Goal: Test GPT-4 Ability To Retrieve Information From Large Context Windows

A fact was placed within a document. GPT-4 (1106-preview) was then asked to retrieve it. The output was evaluated for accuracy. This test was run at 15 different document depths (top > bottom) and 15 different context lengths (1K > 128K tokens). 2x tests were run for larger contexts for a larger sample size.

Prompt engineering

Pressure Testing Claude-2.1 200K via "Needle In A HayStack"

Asking Claude 2.1 To Do Fact Retrieval Across Context Lengths & Document Depth



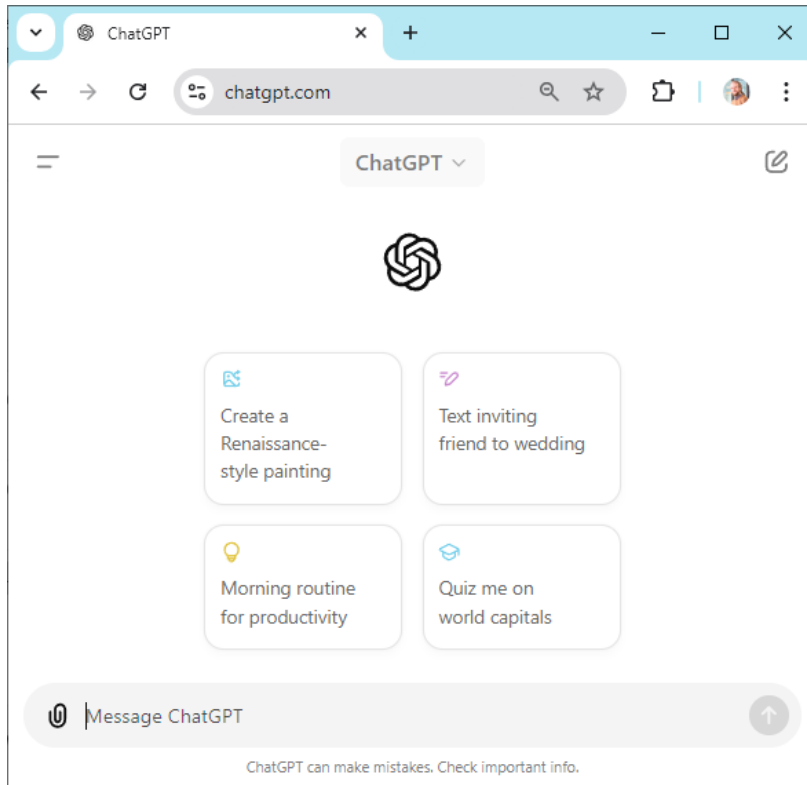
Goal: Test Claude 2.1 Ability To Retrieve Information From Large Context Windows

A fact was placed within a document. Claude 2.1 (200K) was then asked to retrieve it. The output was evaluated (with GPT-4) for accuracy. This test was run at 35 different document depths (top > bottom) and 35 different context lengths (1K > 200K tokens). Document Depths followed a sigmoid distribution

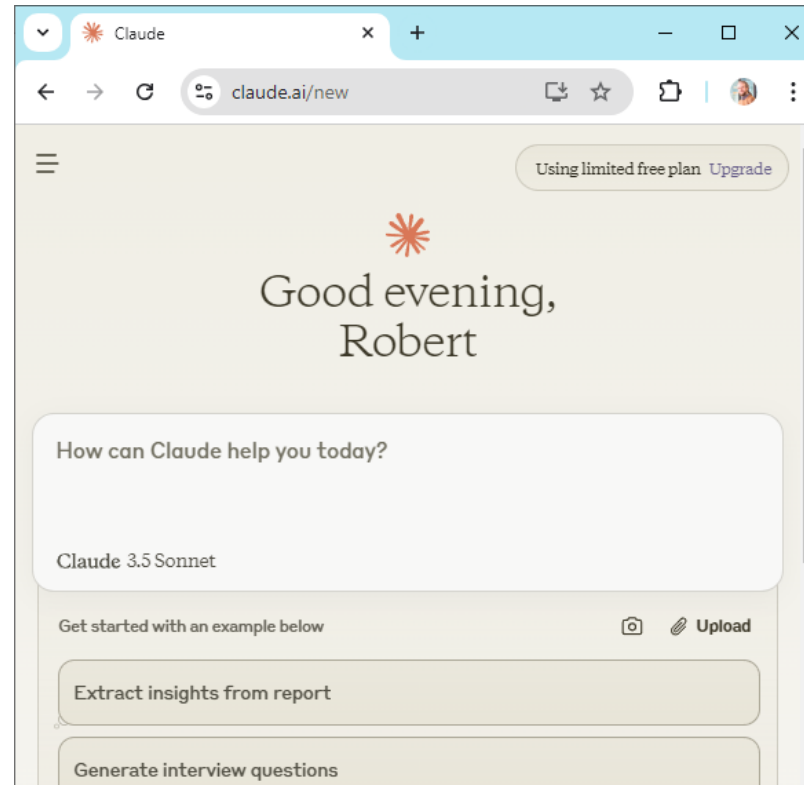
LLMs are everywhere

LLMs are everywhere

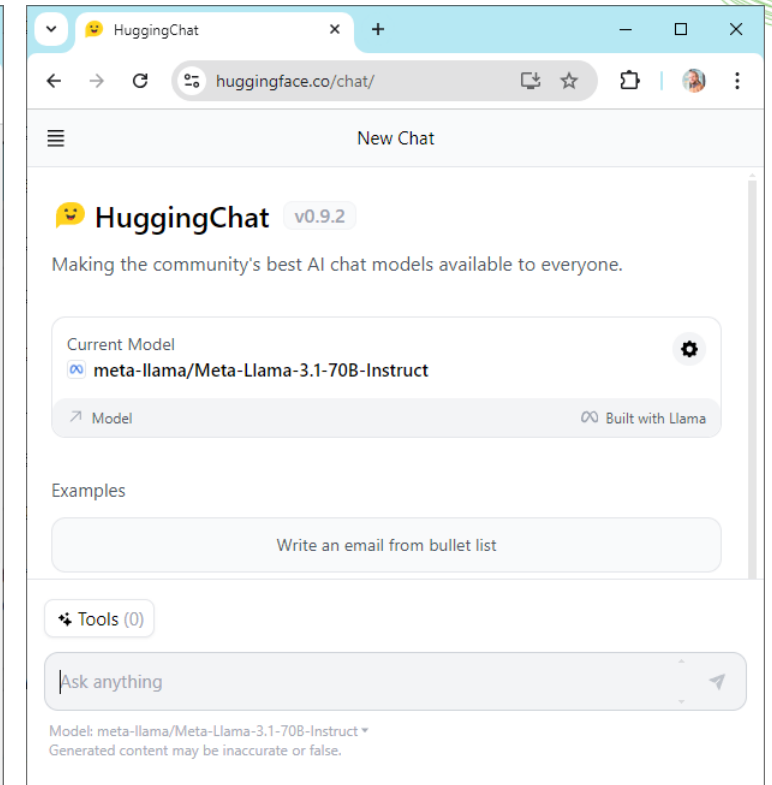
ChatGPT



Claude

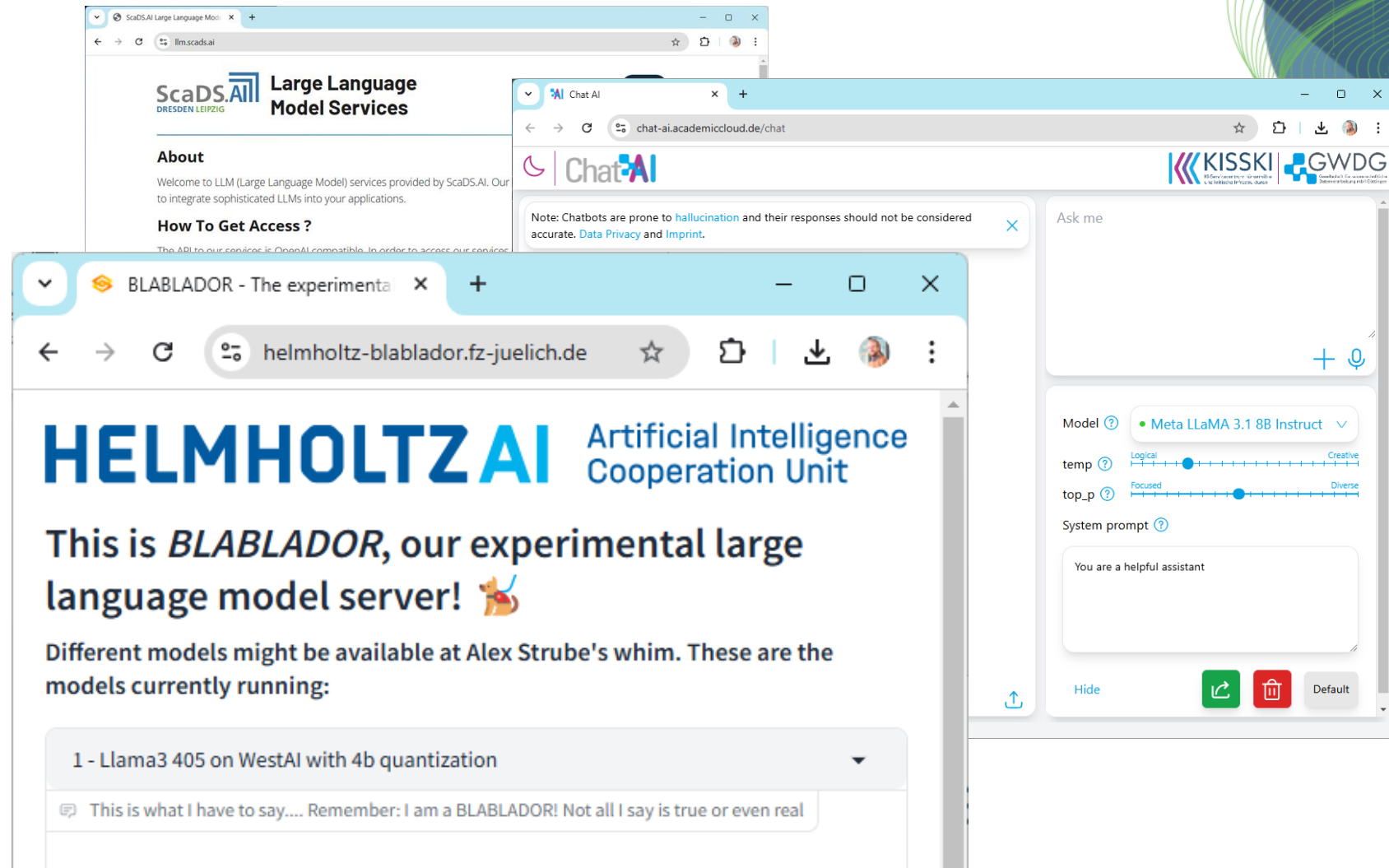


Open models via Huggingface



Institutional AI Infrastructure

- Many institutes have a local LLM Server
 - Great for teaching
 - Privacy preserving
- Cross-institutional infrastructure, in Germany:
 - KISSKI ChatAI
 - Helmholtz Blablador



Commercial AI Infrastructure

Github Models, OpenRouter, Huggingface,...

If it's for free, you are part of the product.

The screenshot shows the OpenRouter website interface. On the left, there are filters for 'Input Modalities' (Text, Image, File), 'Context length' (4K, 64K, 1M), and 'Prompt pricing' (FREE, \$0.5, \$10+). The main section displays a list of models. The first model is 'DeepSeek: Deepseek R1 0528 Qwen3 8B (free)' with 1.26B tokens. Below it, a description states: 'DeepSeek -R1-0528 is a lightly upgraded release of DeepSeek R1 that taps more compute and smarter post-training tricks, pushing its reasoning and inference to the brink of flagship models ...'. It also shows '131K context', '\$0/M input tokens', and '\$0/M output tokens'. The second model is 'DeepSeek: R1 0528 (free)' with 34.1B tokens, described as a 'May 28th update to the original DeepSeek R1 Performance on par with OpenAI o1, but open-sourced and with fully open reasoning tokens. It's 671B parameters in size, with 37B active in an...'. It shows '164K context', '\$0/M input tokens', and '\$0/M output tokens'.

The screenshot shows the GitHub Marketplace website interface. The main heading is 'Enhance your workflow with extensions' with the subtext 'Tools from the community and partners to simplify tasks and automate processes'. Below this is a search bar with 'type:models' entered. The left sidebar shows a navigation menu with 'Featured', 'Copilot', 'Models', 'Catalog', 'Playground', 'Apps', and 'Actions'. The main content area shows a list of models. The first model is 'OpenAI GPT-4.1' by Azure OpenAI Service, with a description: 'gpt-4.1 outperforms gpt-4o across the board, with major gains in coding, instruction following, and lon...'. The second model is 'OpenAI GPT-4o' by Azure OpenAI Service, with a description: 'OpenAI's most advanced multimodal model in the gpt-4o family. Can handle both text and image inputs.'.

Local AI infrastructure

For Small Language Models only

```
Command Prompt - ollama n x + v
C:\Users\rober>ollama run llama3.2:1b
>>> What Python libraries can be used for cell segmentation in microscopy images?
There are several Python libraries that can be used for cell segmentation, but here are
the most popular ones:

1. **OpenCV**: OpenCV is a powerful library for computer vision tasks, including image
segmentation. It provides algorithms such as thresholding, edge detection, and region
growing.
2. **scikit-image**: scikit-image is a library for image processing and analysis. It can be used for
segmentation, and analysis of images. It can be used for thresholding, edge detection, and
watershed transform.
3. **PyTorch**: PyTorch is a popular deep learning framework. It can be used for image
segmentation, and analysis of images. It can be used for thresholding, edge detection, and
watershed transform.
4. **TensorFlow**: TensorFlow is another popular deep learning framework. It can be used for
image segmentation, and analysis of images. It can be used for thresholding, edge detection, and
watershed transform.
5. **scikit-learn**: scikit-learn is a library for machine learning. It can be used for
image segmentation, and analysis of images. It can be used for thresholding, edge detection, and
watershed transform.
6. **Keras**: Keras is a high-level neural networks API. It can be used for image
segmentation, and analysis of images. It can be used for thresholding, edge detection, and
watershed transform.
```

```
Command Prompt - ollama n x + v
Example code for cell segmentation using OpenCV:
```python
import cv2
import numpy as np

Load the image
img = cv2.imread('image.jpg')

Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

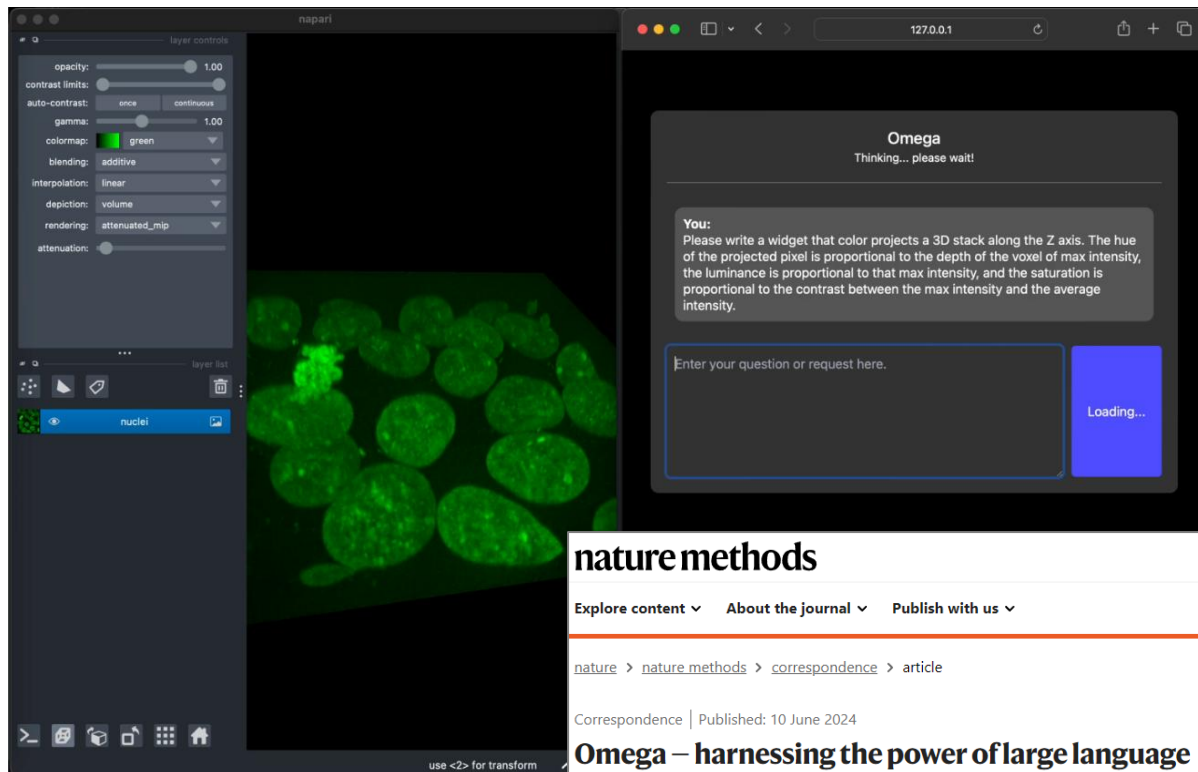
Apply thresholding to segment cells
thresh = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)[1]

Display the segmented image
cv2.imshow('Segmented Image', thresh)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

This code loads an image, converts it to grayscale, applies thresholding using Otsu's method, and displays the
segmented image.
```


LLMs are everywhere

Napari-chatGPT / Omega



<https://github.com/royerlab/napari-chatgpt>
<https://www.nature.com/articles/s41592-024-02310-w>

nature methods

Explore content ▾ About the journal ▾ Publish with us ▾

[nature](#) > [nature methods](#) > [correspondence](#) > article

Correspondence | Published: 10 June 2024

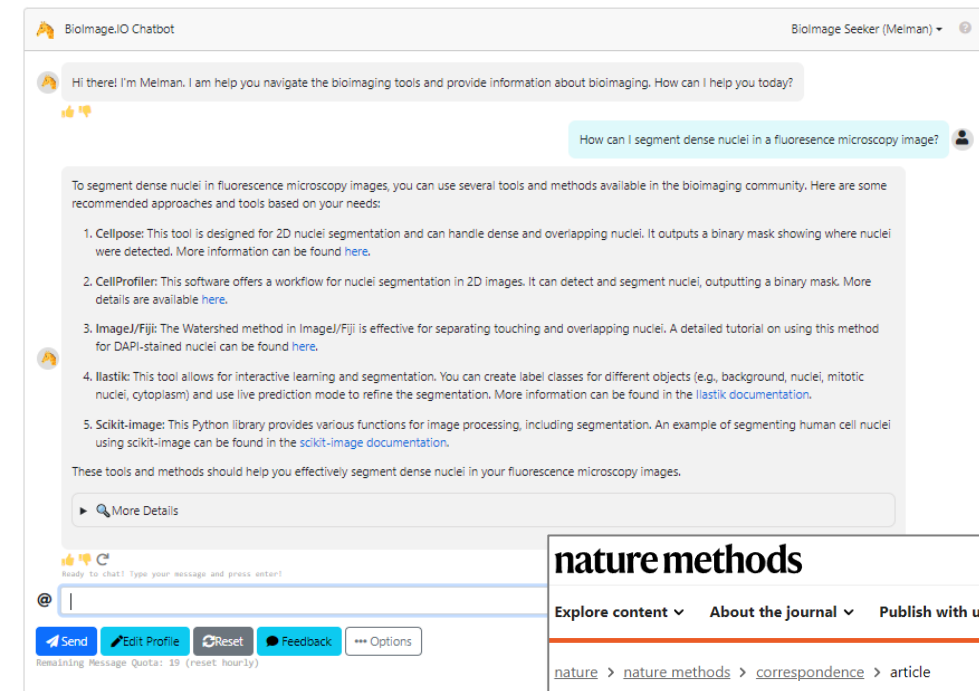
Omega – harnessing the power of large language models for bioimage analysis

Loïc A. Royer

[Nature Methods](#) (2024) | [Cite this article](#)

58 Altmetric | [Metrics](#)

Bioimage-io ChatBot



<https://bioimage.io/chat/>
<https://www.nature.com/articles/s41592-024-02370-y>

nature methods

Explore content ▾ About the journal ▾ Publish with us ▾

[nature](#) > [nature methods](#) > [correspondence](#) > article

Correspondence | Published: 09 August 2024

BioImage.IO Chatbot: a community-driven AI assistant for integrative computational bioimaging

Wanlu Lei, Caterina Fuster-Barceló, Gabriel Reder, Arrate Muñoz-Barrutia & Wei Ouyang

[Nature Methods](#) **21**, 1368–1370 (2024) | [Cite this article](#)

865 Accesses | 1 Altmetric | [Metrics](#)

LLMs are everywhere

aider

```
macbook$ aider demo.py
Added demo.py to the chat
Using git repo: .git

demo.py> add a name param to the `greeting` function. add all the types.

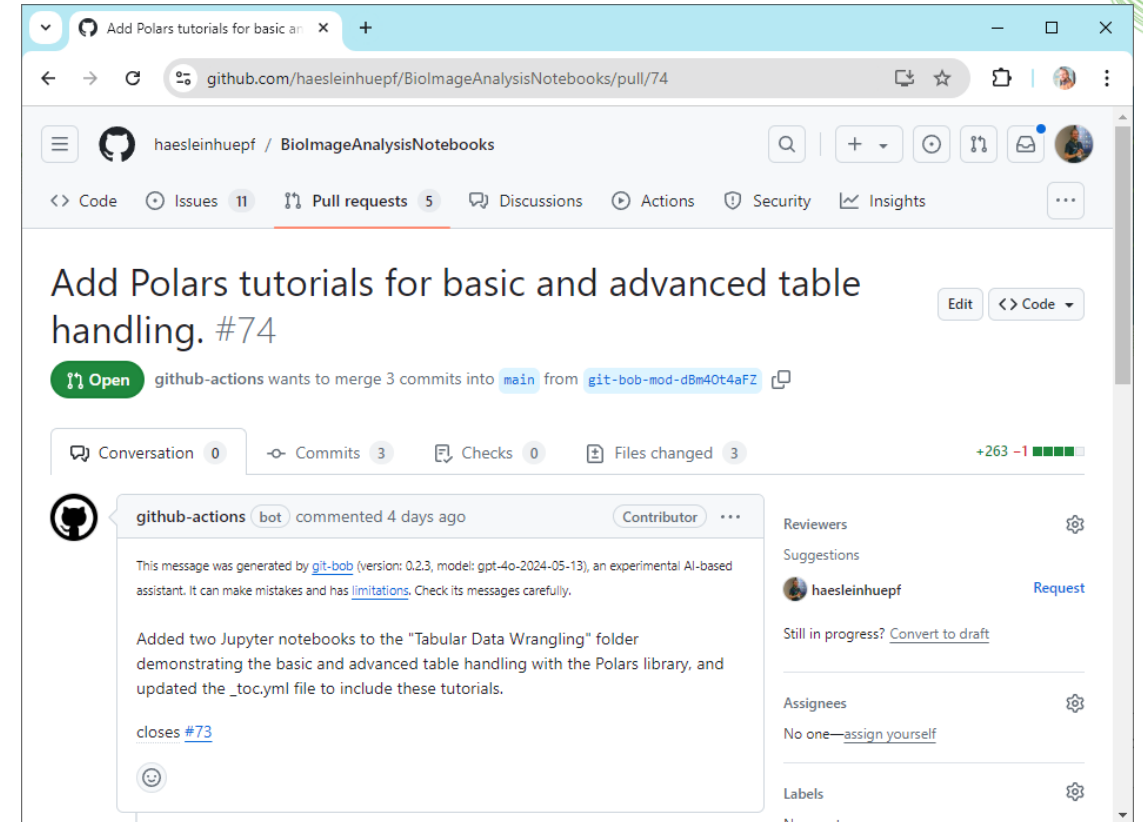
I will update the greeting function to accept a name parameter and include type
hints for the parameter and the return type.

demo.py
<<<<<< ORIGINAL
def greeting():
    print("Hello, world!")
=====
def greeting(name: str) -> None:
    print(f"Hello, {name}!")
>>>>>> UPDATED

Applied edit to demo.py
Commit 9237455 aider: Updated greeting function to accept a name parameter and
include type hints.

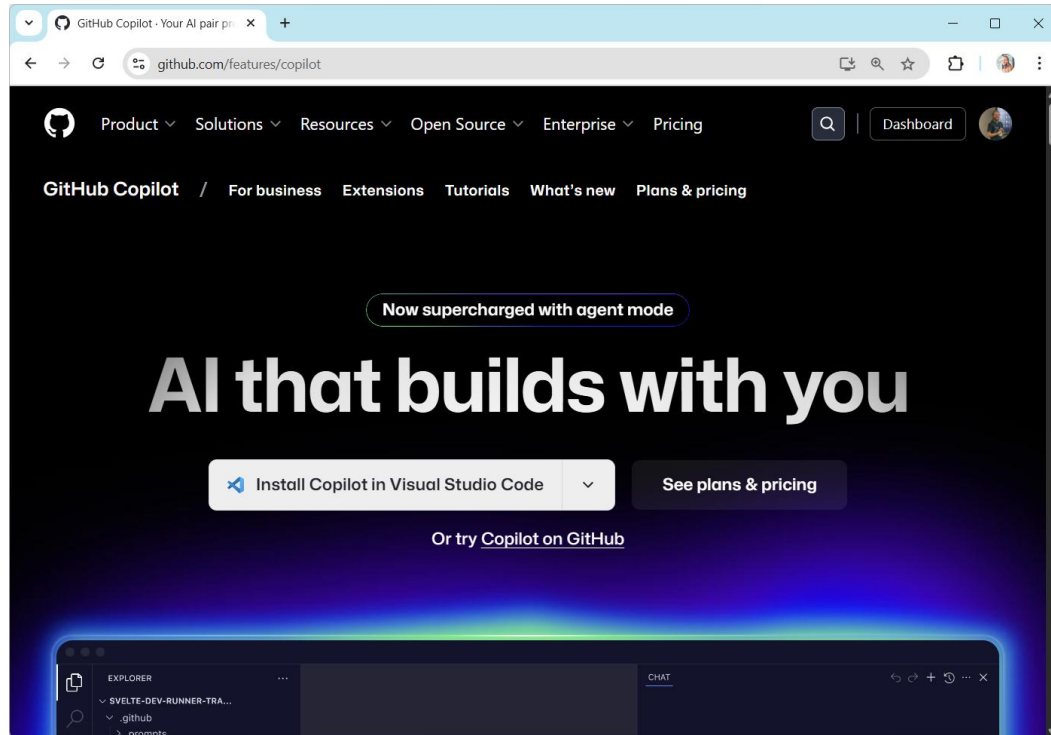
demo.py>
```

git-bob

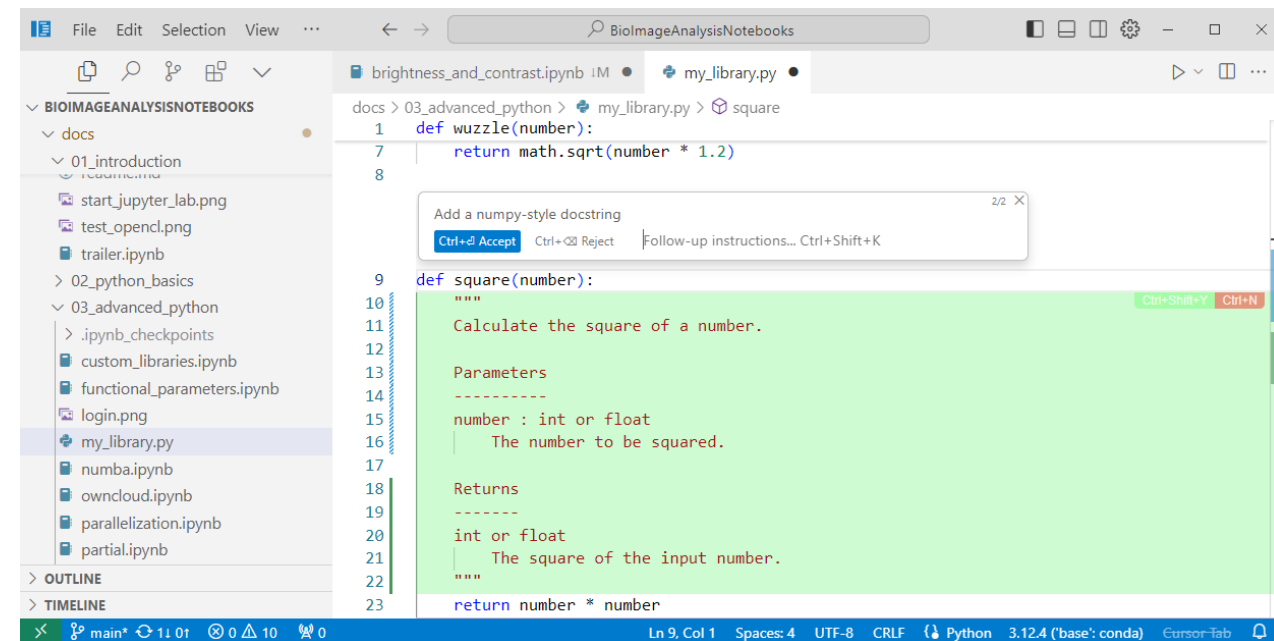


LLMs are everywhere

Github Copilot



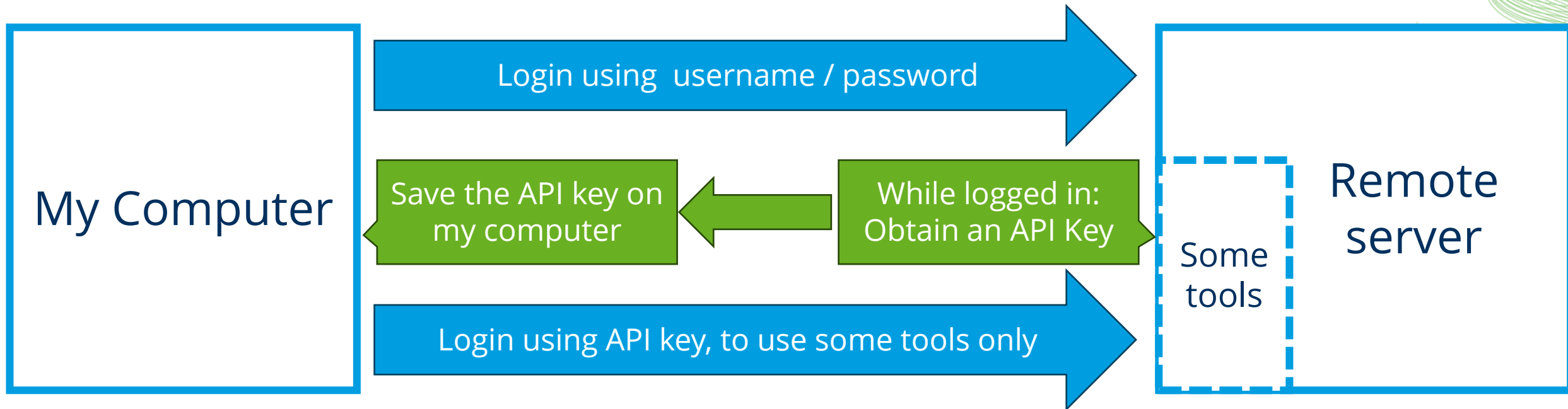
Cursor



Usings LLMs in Python

Authentication using API-Keys

Application programming interface (API) keys: **A modern kind of password** basically invented to avoid entering username/passwords over and over again



Never share your API key with others

API key can be revoked / invalidated any time,

The OpenAI Application Programming Interface

De-facto standard for communicating with LLMs

```
[1]: import os
import openai
openai.__version__
```

```
[1]: '1.5.0'
```

```
[2]: client = openai.OpenAI()
client
```

```
[2]: <openai.OpenAI at 0x1bd8871ec90>
```

Ollama

```
client = openai.OpenAI(
    base_url = "http://localhost:11434/v1",
    api_key = "none" # not required by ollama
)
```

KISSKI

```
client = openai.OpenAI(
    base_url="https://chat-ai.academiccloud.de/v1",
    api_key=os.environ.get('KISSKI_API_KEY')
)
```


The OpenAI Application Programming Interface

Request

```
my_messages = [{  
    "role": "user",  
    "content": "What's the capital of France?"  
}]  
my_messages
```

```
[{'role': 'user', 'content': "What's the capital of France?"}]
```

```
response = client.chat.completions.create(  
    model="gpt-4o",  
    messages=my_messages  
)  
response
```

Response

```
ChatCompletion(id='chatcmpl-BeGoMYWRYYYQ3XvySyZGRvcqDFCCT', choices=[Choice(finish_reason='stop', index=0, logprobs=None, message=ChatCompletionMessage(content='The capital of France is Paris.', refusal=None, role='assistant', annotations=[], audio=None, function_call=None, tool_calls=None))], created=1748937442, model='gpt-4o-2024-08-06', object='chat.completion', service_tier='default', system_fingerprint='fp_07871e2ad8', usage=CompletionUsage(completion_tokens=7, prompt_tokens=13, total_tokens=20, completion_tokens_details=CompletionTokensDetails(accepted_prediction_tokens=0, audio_tokens=0, reasoning_tokens=0, rejected_prediction_tokens=0), prompt_tokens_details=PromptTokensDetails(audio_tokens=0, cached_tokens=0)))
```

```
response.choices[0].message.content
```

```
'The capital of France is Paris.'
```

Common helper functions

To make our life simpler, we use such helper functions.

```
def prompt_openai(prompt:str, model="gpt-4o"):
    """A prompt helper function that sends a prompt to openAI
    and returns only the text response.
    """
    # setup connection to the LLM
    client = openai.OpenAI()

    # submit prompt
    response = client.chat.completions.create(
        model=model,
        messages=[{"role": "user", "content": prompt}]
    )

    # extract answer
    return response.choices[0].message.content
```

```
prompt_openai("How many o are in Woolloomooloo?")
```

```
'The word "Woolloomooloo" contains eight \'o\'s.'
```

```
[2]: def prompt_ollama(prompt:str, model="gemma3:1b"):
    """A prompt helper function that sends a prompt to
    ollama and returns only the text response."""
    # setup connection to the LLM server
    client = openai.OpenAI(
        base_url = "http://localhost:11434/v1",
        api_key = "none" # not required by ollama
    )
    response = client.chat.completions.create(
        model=model,
        messages=[{"role": "user", "content": prompt}]
    )

    # extract answer
    return response.choices[0].message.content
```

```
[3]: prompt_ollama("Hi!")
```

```
[3]: "Hi there! How's your day going so far? 😊 \n\nIs there
anything you'd like to talk about or any I can help you w
ith?"
```

Common helper functions

To make our life simpler, we use such helper functions.

```
[2]: def prompt_kisski(prompt:str, model="meta-llama-3.1-70b-instruct"):
    """A prompt helper function that sends a message to KISSKI Chat AI API
    and returns only the text response.
    """
    # setup connection to the LLM-server
    client = openai.OpenAI(
        base_url="https://chat-ai.academiccloud.de/v1",
        api_key=os.environ.get('KISSKI_API_KEY')
    )

    response = client.chat.completions.create(
        model=model,
        messages= [{"role": "user", "content": prompt}]
    )

    # extract answer
    return response.choices[0].message.content
```

```
[3]: prompt_kisski("Hi!")
```

```
[3]: "It's nice to meet you. Is there something I can help you with or would you
like to chat?"
```

```
[2]: def prompt_openrouter(prompt:str, model="google/gemini-2.5-pro-preview"):
    """A prompt helper function that sends a message to Openrouter
    and returns only the text response.
    """
    # setup connection to the LLM-server
    client = openai.OpenAI(
        base_url="https://openrouter.ai/api/v1",
        api_key=os.environ.get('OPENROUTER_API_KEY')
    )

    response = client.chat.completions.create(
        model=model,
        messages= [{"role": "user", "content": prompt}]
    )

    # extract answer
    return response.choices[0].message.content
```

```
[3]: prompt_openrouter("Hi!")
```

```
[3]: 'Hi there! How can I help you today?'
```

OpenAI API: Querying models

```
[9]: client = openai.OpenAI()

print("\n".join(sorted([model.id for model in client.models.list().data])))
```

```
babbage-002
chatgpt-4o-latest
codex-mini-latest
computer-use-preview
computer-use-preview-2025-03-11
dall-e-2
dall-e-3
davinci-002
```

```
...
gpt-4.1
gpt-4.1-2025-04-14
gpt-4.1-mini
gpt-4.1-mini-2025-04-14
gpt-4.1-nano
gpt-4.1-nano-2025-04-14
gpt-4.5-preview
gpt-4.5-preview-2025-02-27
gpt-4o
gpt-4o-2024-05-13
gpt-4o-2024-08-06
gpt-4o-2024-11-20
...
```

```
client = openai.OpenAI(
    base_url = "https://helmholtz-blablador.fz-juelich.de:8000/v1",
    api_key = os.environ.get('BLABLADOR_API_KEY')
)

print("\n".join([model.id for model in client.models.list().data]))
```

```
1 - Llama3 405 the best general model and big context size
1 - Ministral 8b - the fast model
1 - Teuken-7B-instruct-research-v0.4 - The OpenGPT-X model
10 Mistral-Nemo-Instruct-2407 - Our fast-experimental - with a large context size
2 - Qwen3 30B A3B - a reasoning model from Alibaba from April 2025
3 - DeepCoder-14B-Preview - the code model from 09.04.2025
alias-code
alias-fast
alias-fast-experimental
alias-large
alias-llama3-huge
alias-opengptx
alias-reasoning
```

Good Scientific Practice

Quiz: What could possibly go wrong?

... when using LLMs for scientific [bio image] data analysis tasks?

Challenges

Generative artificial intelligence imposes a risk to science



Elisabeth Bik @MicrobiomDigest · 16h

The amount of (suspected) AI-generated manuscripts and published papers is sharply rising.

Why are journals not doing a better job screening for these?

This generates an enormous burden for peer reviewers and pollutes the scientific literature.

We need better tools and rules.



19



84



312



26K



The raise of AI-generated science

“the explosion of AI-assisted productivity in published manuscripts (the systematic search strategy used here identified an average of 4 papers per annum from 2014 to 2021, but 190 in 2024–9 October alone)”
Suchak et al 2025



LLMs for reviewing

Association for the Advancement of Artificial Intelligence Launches AI-Powered Peer Review Assessment System

[Washington, DC] — The Association for the Advancement of Artificial Intelligence (AAAI), a leading nonprofit dedicated to advancing scientific research and collaboration, today announced a pilot program that strategically incorporates Large Language Models (LLMs) to enhance the academic paper review process for the AAAI-26 conference. This initiative aims to improve efficiency while maintaining the highest standards of scientific rigor and human oversight.

Enhancing Scientific Review, Not Replacing Human Expertise

The pilot program will thoughtfully integrate LLM technology at two specific points in the established review process:

1. **Supplementary First-Stage Reviews:** LLM-generated reviews will be included as one component of the initial review stage, providing an additional perspective alongside traditional human expert evaluations.
2. **Discussion Summary Assistance:** LLMs will assist the Senior Program Committee (SPC) members by summarizing reviewer discussions, helping to highlight key points of consensus and disagreement among human reviewers.

"This pilot represents a careful, measured approach to incorporating new technology into the scientific review process," said Stephen Smith, AAAI President. "We're exploring how LLMs can complement—not replace—the irreplaceable expertise and judgment of our human reviewers."



For Immediate Release May 16, 2025

LLMs for reviewing

Preserving Human Decision-Making and Scientific Integrity

AAAI emphasizes that this pilot maintains the primacy of human expertise and judgment in several important ways:

1. No Displacement of Human Reviewers: No human reviewers are being replaced at any stage of the process.
2. No Automated Decision-Making: LLM-generated content will not be used for automated accept/reject decisions.
3. No Numerical Ratings from LLMs: The technology will not provide numerical scores or ratings for papers.
4. Human Oversight at All Stages: Human experts will review all LLM-generated content.

Cutting-Edge Methods with Rigorous Safeguards

The pilot employs state-of-the-art methodologies in the responsible deployment of LLM technology, including:

1. Multi-step reasoning processes at inference time
2. Web search capabilities as tools in the reasoning chain
3. Rigorous checks for proper data source attribution
4. Comprehensive monitoring and evaluation of LLM contributions

Stringent Privacy and Data Protection

AAAI has implemented strict privacy controls throughout the pilot:

1. All paper data and reviewer information will remain confidential
2. No data will be used for LLM training
3. Information will not be repurposed for any use beyond generating the intended reviews and summaries
4. Complete compliance with all data protection regulations



For Immediate Release May 16, 2025

Rules...

„When making their results publicly available, researchers should, in the spirit of research integrity, disclose whether or not they have used generative models, and if so, which ones, for what purpose and to what extent.“

Statement by the Executive Committee
of the Deutsche Forschungsgemeinschaft (DFG,
German Research Foundation) on the Influence of
Generative Models of Text and Image Creation on
Science and the Humanities and on the DFG's
Funding Activities

September 2023

Good scientific practice

If you use custom code written by ...

a human expert

an expert LLM

You should ...

- Understand the code (roughly)
- Question used methods
- Check results carefully
- Test code on samples the expert didn't see



Good scientific practice

If you use custom code written by ...

a human expert

an expert LLM

You should ...

- Pay the expert
- Mention the expert
- Share responsibility
- Ask the expert endless questions
- Share how you prompted the expert



\$100/h



co-author



\$0.1/h



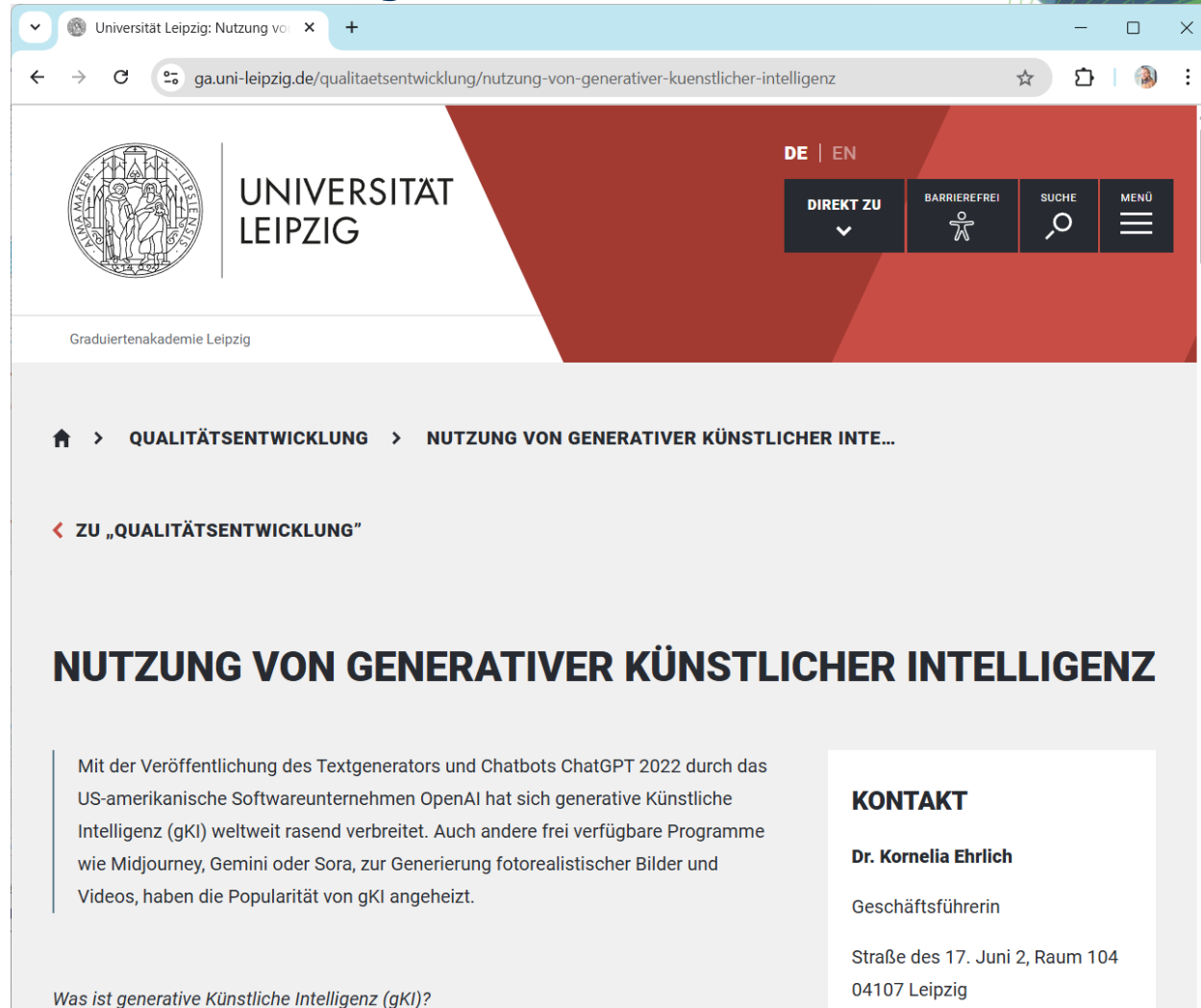
in methods



Guidelines at Leipzig University

... are in the making.

Check the website of the Graduate Academy in the meantime.



Learn more...

YouTube DE

308 - An introduction to language models with focus on GPT

With a special focus on GPT

308

@DigitalSreeni

DigitalSreeni
110K subscribers

Subscribe

232

Share

5.6K views 1 year ago

Video 308: An introduction to language models, With a special focus on GPT

<https://www.youtube.com/watch?v=9Y7f4j396hl>

YouTube DE

How Large Language Models Work

Why LLMs Matter

IBM T...
888K...

Subscribe

9.8K

Share

637K views 1 year ago #largelanguagemodel #GenerativeAI #llm

Learn in-demand Machine Learning skills now → <https://ibm.biz/BdK65D>

Learn about watsonx → <https://ibm.biz/BdvxRj>

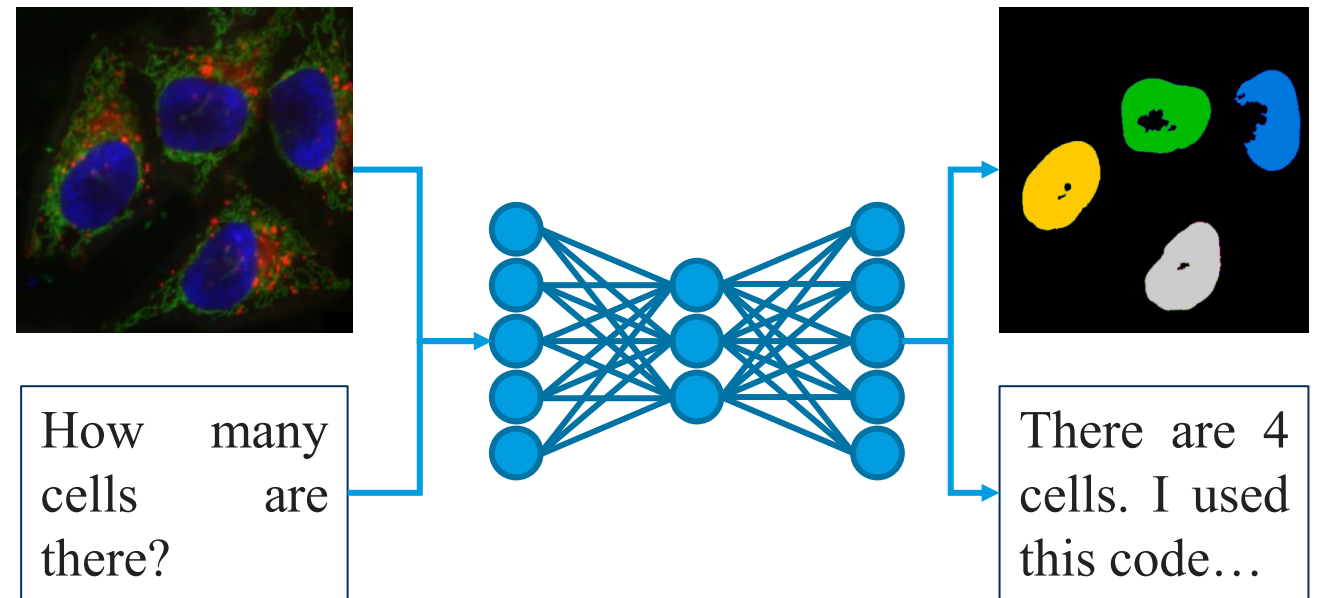
<https://www.youtube.com/watch?v=5sLYAQS9sWQ>

Summary & outlook

- Transformers / LLMs disrupt work in couple of fields
 - Text production
 - Programming
- Prompt engineering becomes a key skill in many disciplines

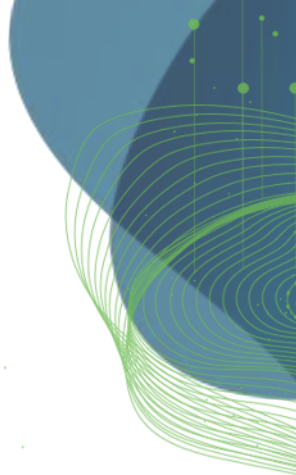
Coming up next:

- Code Generation, Multi-modal LLMs and agentic systems analyzing bio-imaging data



Exercises

Robert Haase



Option 1: Use Ollama (locally on your PC)

Download and install Ollama

<https://ollama.com/>

Download a model, e.g. using

```
ollama run gemma3:1b
```

Or

```
ollama run gemma3:4b
```

Or

```
ollama run llama3.2:1b
```

Start the server using (linux)

```
ollama serve
```

```
Command Prompt - ollama run gemma:2b

(tea2024) C:\structure\code\BIDS-lecture-2024\06_chatbots>ollama run gemma:2b
>>> What's the capital of France?
The capital of France is Paris. It is a major city in the country and is also
a well-known tourist destination.

>>> Send a message (/? for help)
```

```
Command Prompt - ollama serve

time=2024-05-06T17:45:20.995+02:00 level=INFO source=images.go:813 msg="total u
nused blobs removed: 0"
time=2024-05-06T17:45:20.996+02:00 level=INFO source=routes.go:1110 msg="Listen
ing on 127.0.0.1:11434 (version 0.1.29)"
time=2024-05-06T17:45:20.996+02:00 level=INFO source=payload_common.go:112 msg=
"Extracting dynamic libraries to C:\\Users\\haase\\AppData\\Local\\Temp\\ollama
1382644311\\runners ..."
time=2024-05-06T17:45:21.032+02:00 level=INFO source=payload_common.go:139 msg=
"Dynamic LLM libraries [cpu_avx2 rocm_v5.7 cuda_v11.3 cpu cpu_avx]"
```


Option 2: Get a Helmholtz Blablador API Key

Yes, this is free for German academics.

The image displays two screenshots of the Helmholtz Codebase - GitLab interface. The left screenshot shows the login page with a green arrow pointing to the 'Sign in with Helmholtz ID' button. The right screenshot shows the user profile dropdown menu with a green arrow pointing to the 'Sign out' option.

Helmholtz Codebase - GitLab
Provided by HIFIS for all of Helmholtz & Partners

Login: Please sign in with [Helmholtz ID](#) (also known as Helmholtz AAI). Select your home institution or a social provider like ORCID, GitHub, Google. Active HZDR employees can use the login form on this page.

Support: If you have problems signing in, please contact us via support@hifis.net or support.hifis.net.

Documentation: <https://hifis.net/doc/software/gitlab/getting-started/>

Sign in with Helmholtz ID
☐ Login merken
oder melde dich an mit

HZDR

Nutzername

Welcome to GitLab
Faster releases. Better code. Less pain.

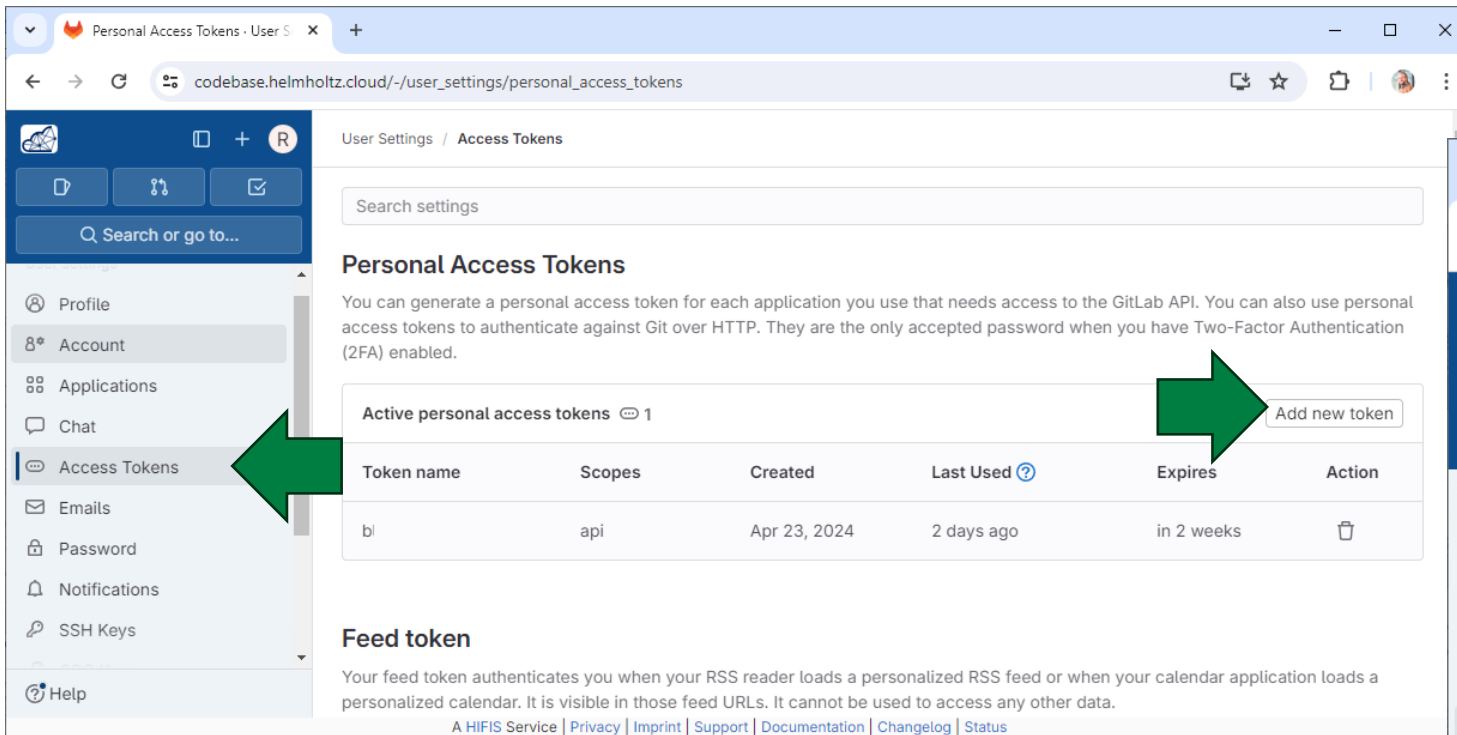
Explore public projects
Public projects are an easy way to allow everyone to have read-only

Robert Haase
@robert.haase

- Set status
- Edit profile
- Preferences
- Sign out
- Merge requests
- To-Do List

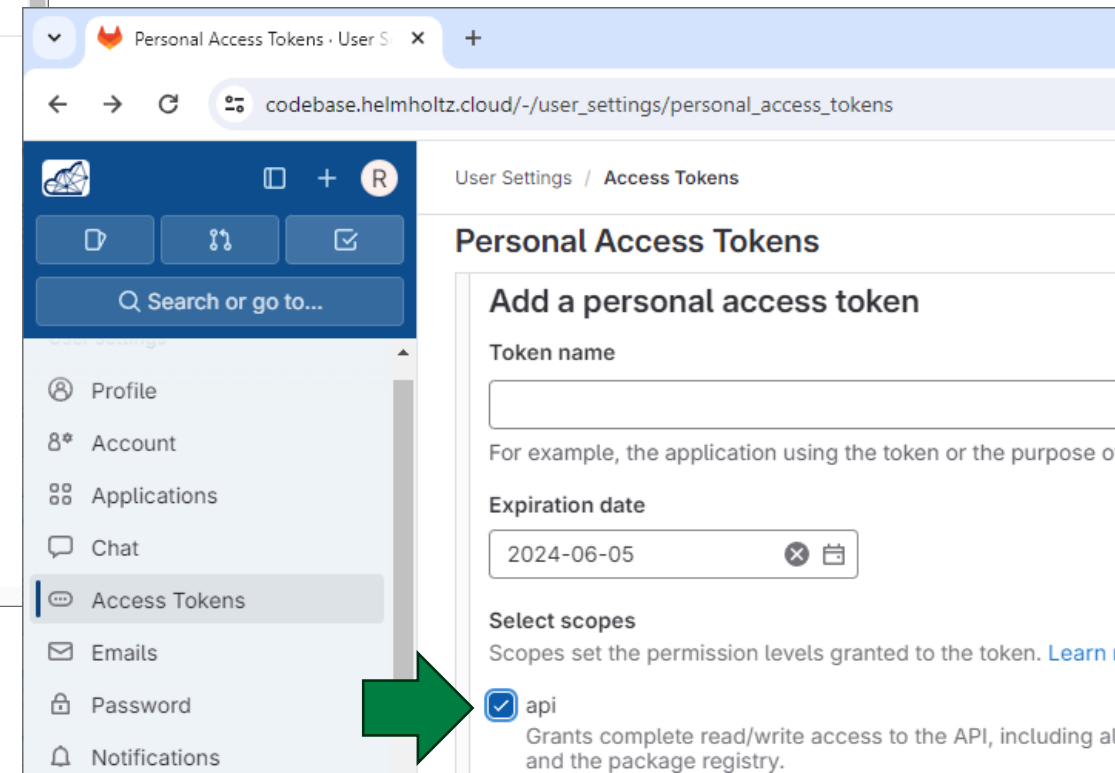
Option 2: Get a Helmholtz Blablador API Key

This is free for German academics.



The screenshot shows the 'Personal Access Tokens' page in the Helmholtz Blablador user settings. The left sidebar contains a menu with options: Profile, Account, Applications, Chat, Access Tokens (highlighted with a green arrow), Emails, Password, Notifications, SSH Keys, and Help. The main content area has a search bar and a section titled 'Personal Access Tokens' with a description. Below this, there is a table of 'Active personal access tokens' with one token listed. A green arrow points to the 'Add new token' button.

| Token name | Scopes | Created | Last Used | Expires | Action |
|------------|--------|--------------|------------|------------|--------|
| bl | api | Apr 23, 2024 | 2 days ago | in 2 weeks | |



The screenshot shows the 'Add a personal access token' form in the Helmholtz Blablador user settings. The left sidebar is the same as the previous screenshot, with 'Access Tokens' highlighted. The main content area shows the form with fields for 'Token name', 'Expiration date' (set to 2024-06-05), and 'Select scopes'. The 'api' scope is selected with a green arrow.

Add a personal access token

Token name

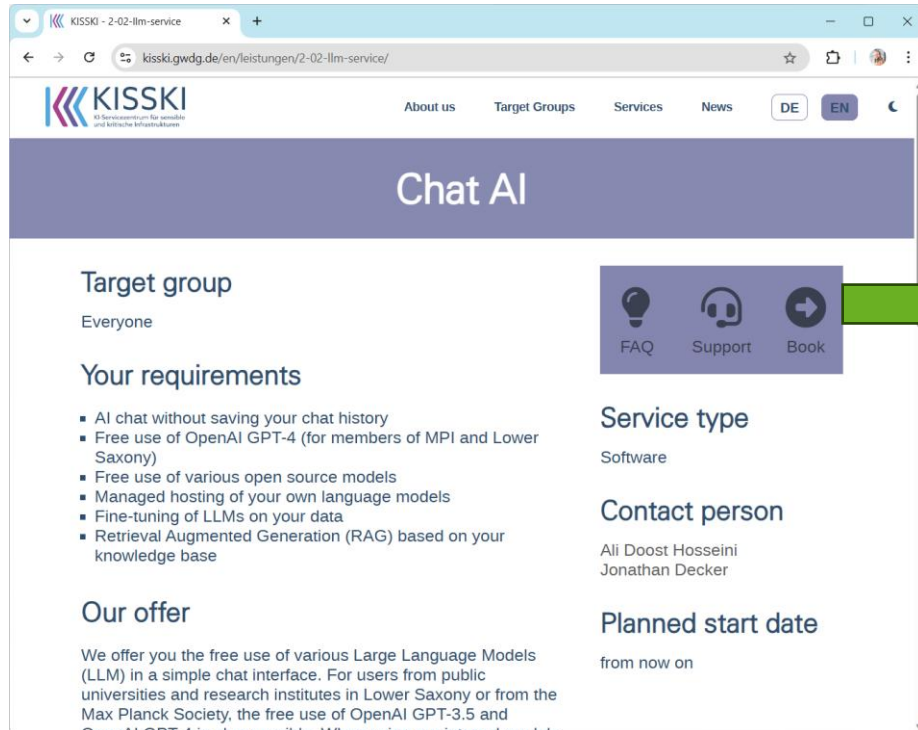
Expiration date: 2024-06-05

Select scopes

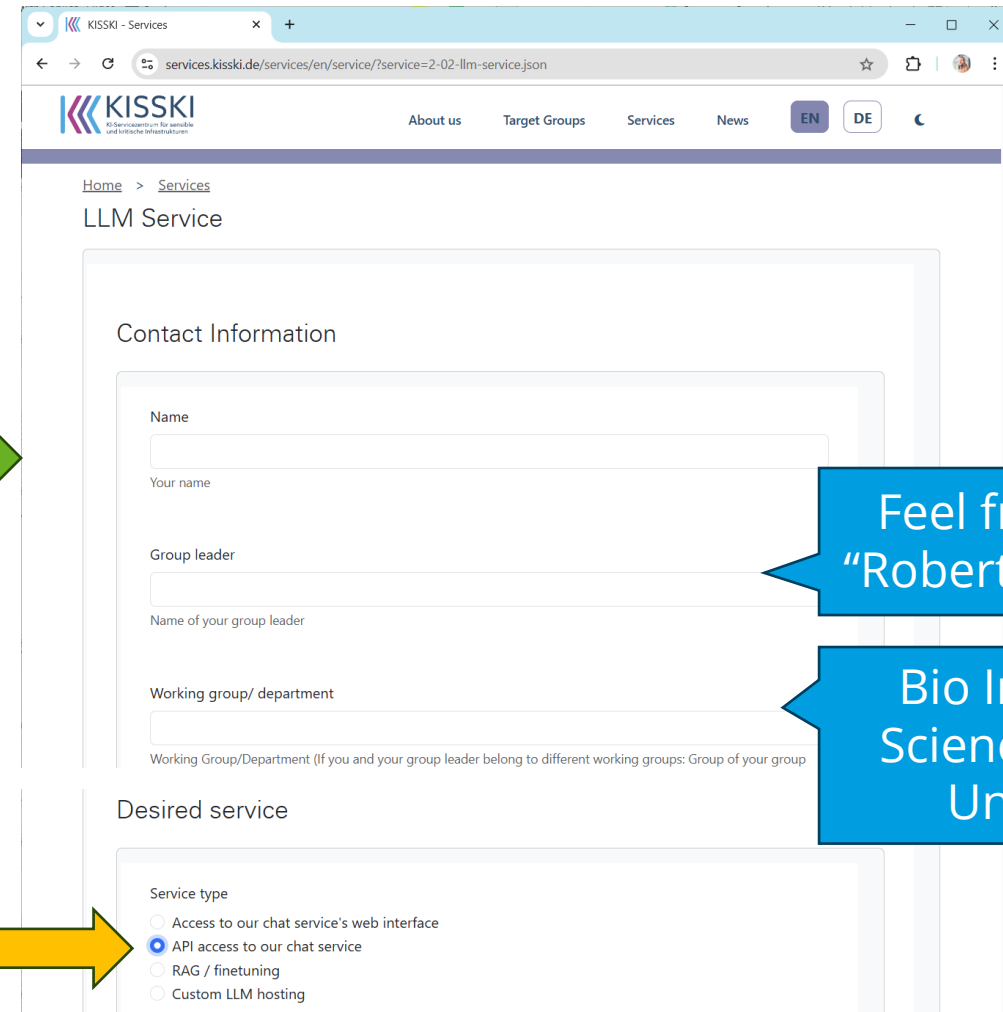
☒ api

Option 3: Get a GWDG KISSKI API Key

This is free for German academics.



The screenshot shows the KISSKI Chat AI service page. The header includes the KISSKI logo and navigation links: About us, Target Groups, Services, News, and language toggles for DE and EN. The main heading is "Chat AI". Below it, the "Target group" is listed as "Everyone". The "Your requirements" section includes a bulleted list: AI chat without saving your chat history, Free use of OpenAI GPT-4 (for members of MPI and Lower Saxony), Free use of various open source models, Managed hosting of your own language models, Fine-tuning of LLMs on your data, and Retrieval Augmented Generation (RAG) based on your knowledge base. The "Our offer" section states: "We offer you the free use of various Large Language Models (LLM) in a simple chat interface. For users from public universities and research institutes in Lower Saxony or from the Max Planck Society, the free use of OpenAI GPT-3.5 and OpenAI GPT-4 is also possible. When using our internal models...". On the right side, there are icons for FAQ, Support, and Book, and a green arrow points from the "Book" icon to the next screenshot.



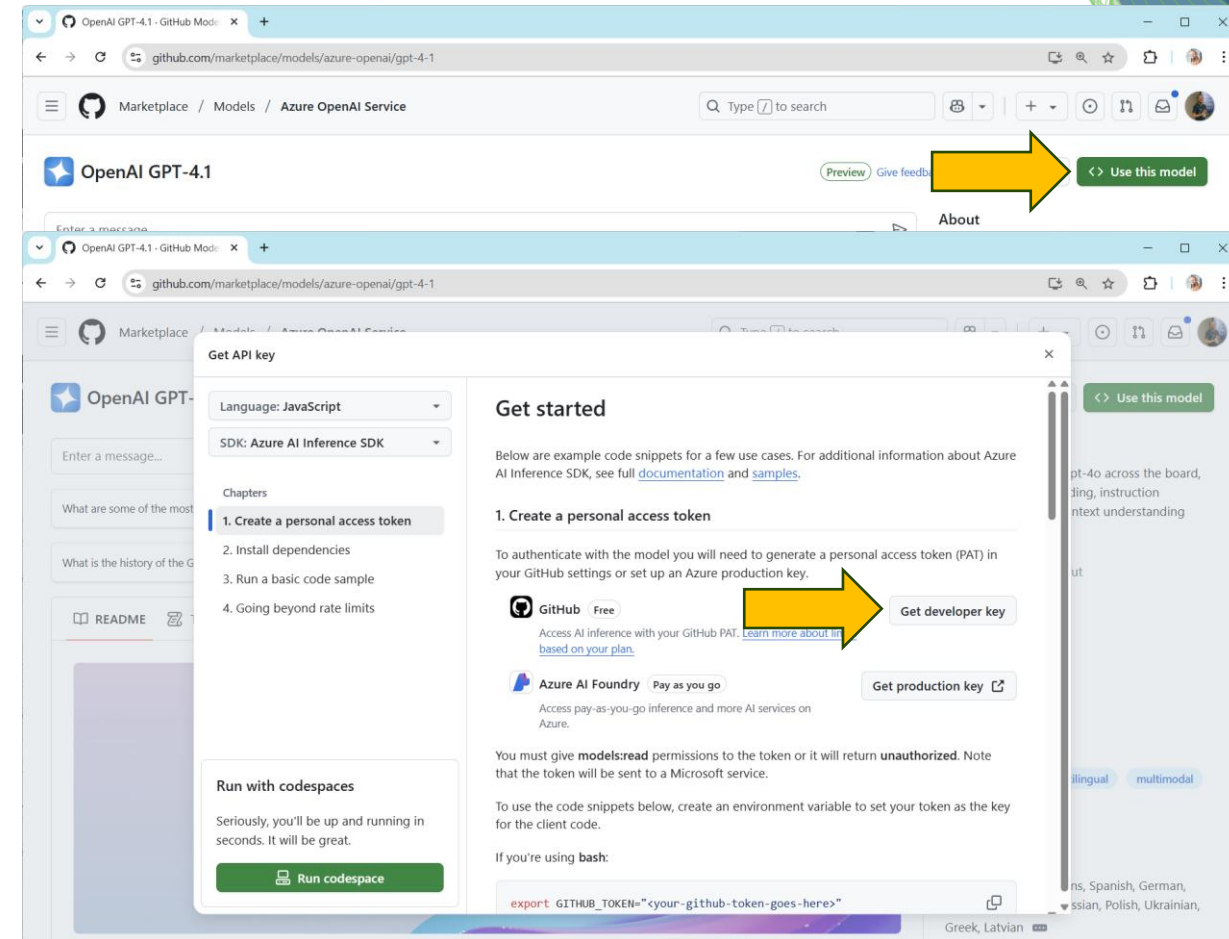
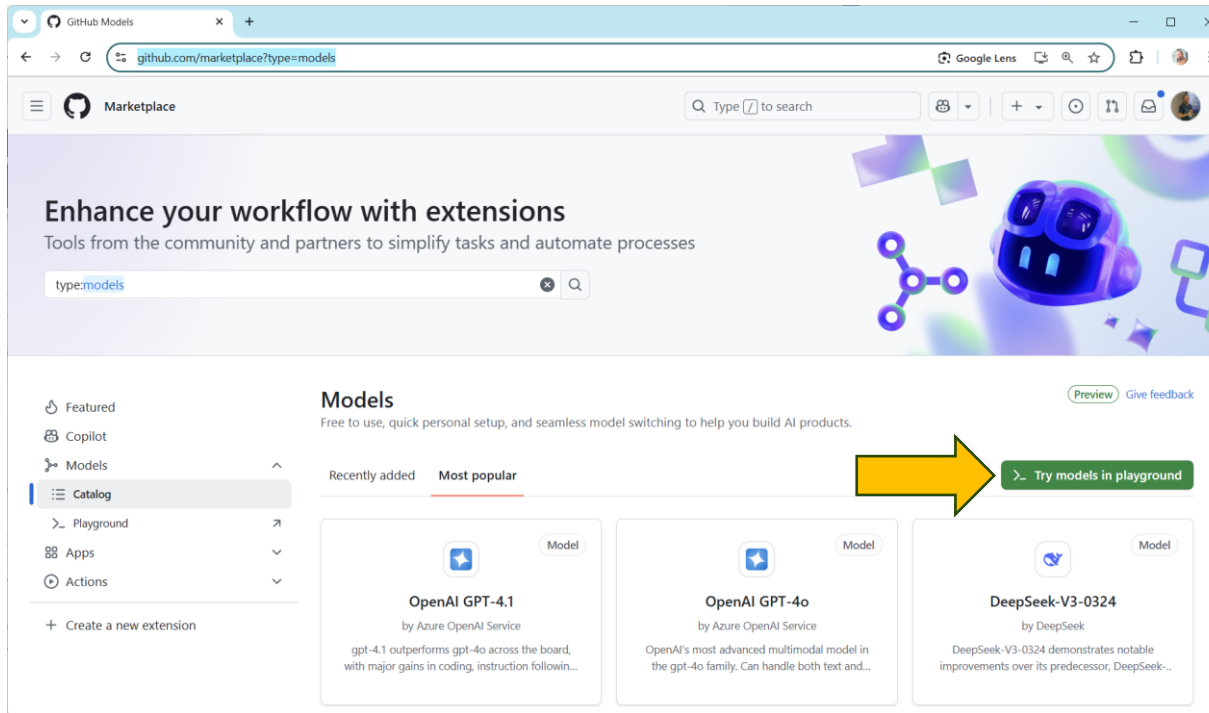
The screenshot shows the KISSKI LLM Service registration form. The header includes the KISSKI logo and navigation links: About us, Target Groups, Services, News, and language toggles for EN and DE. The main heading is "LLM Service". Below it, the "Contact Information" section includes fields for Name, Your name, Group leader, Name of your group leader, Working group/ department, and Working Group/Department (If you and your group leader belong to different working groups: Group of your group). The "Desired service" section includes a "Service type" dropdown menu with options: Access to our chat service's web interface, API access to our chat service (selected), RAG / finetuning, and Custom LLM hosting. A yellow arrow points from the "Book" icon in the previous screenshot to the "API access to our chat service" option.

Feel free to enter
"Robert Haase" here

Bio Image Data
Science Lecture /
Uni Leipzig

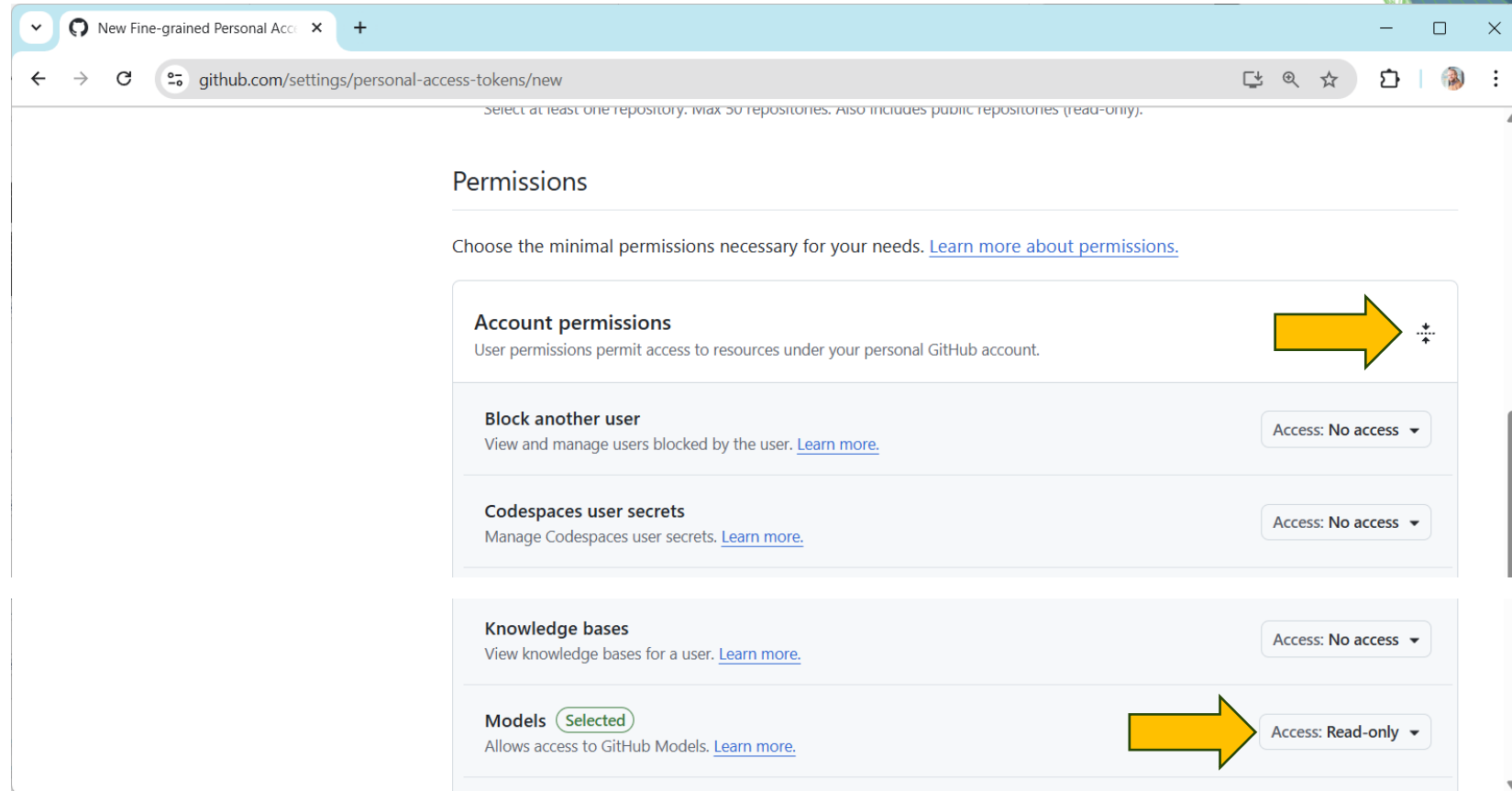
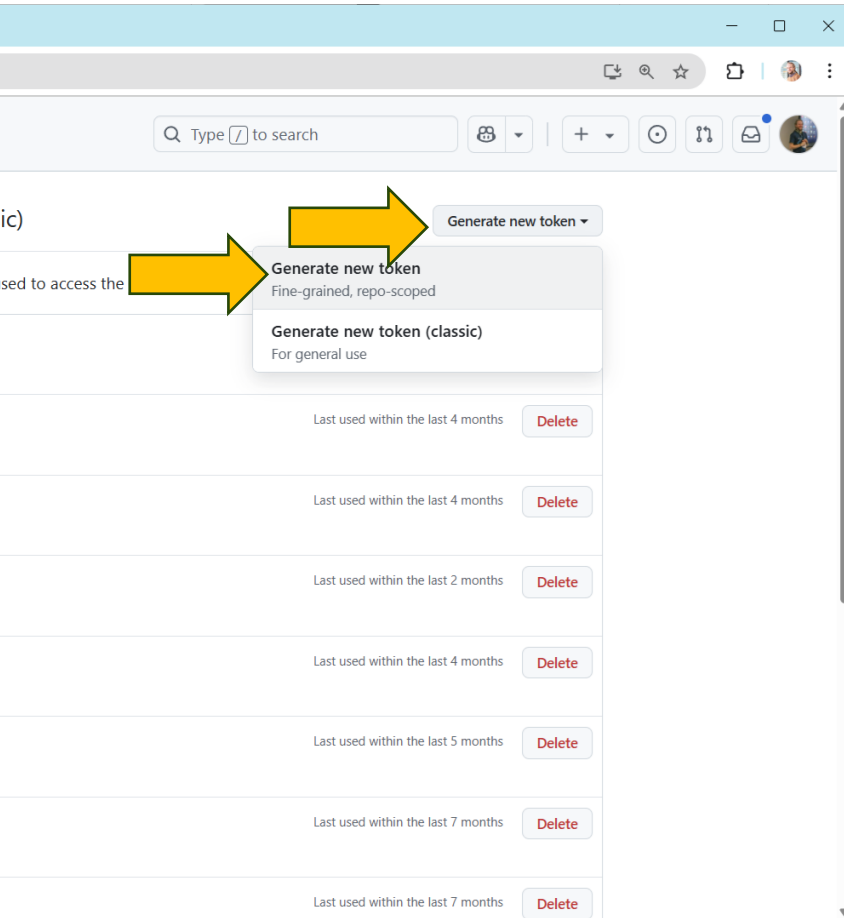
Option 4: Github Models Marketplace

This is a free commercial service. "If it's free you're part of the product."



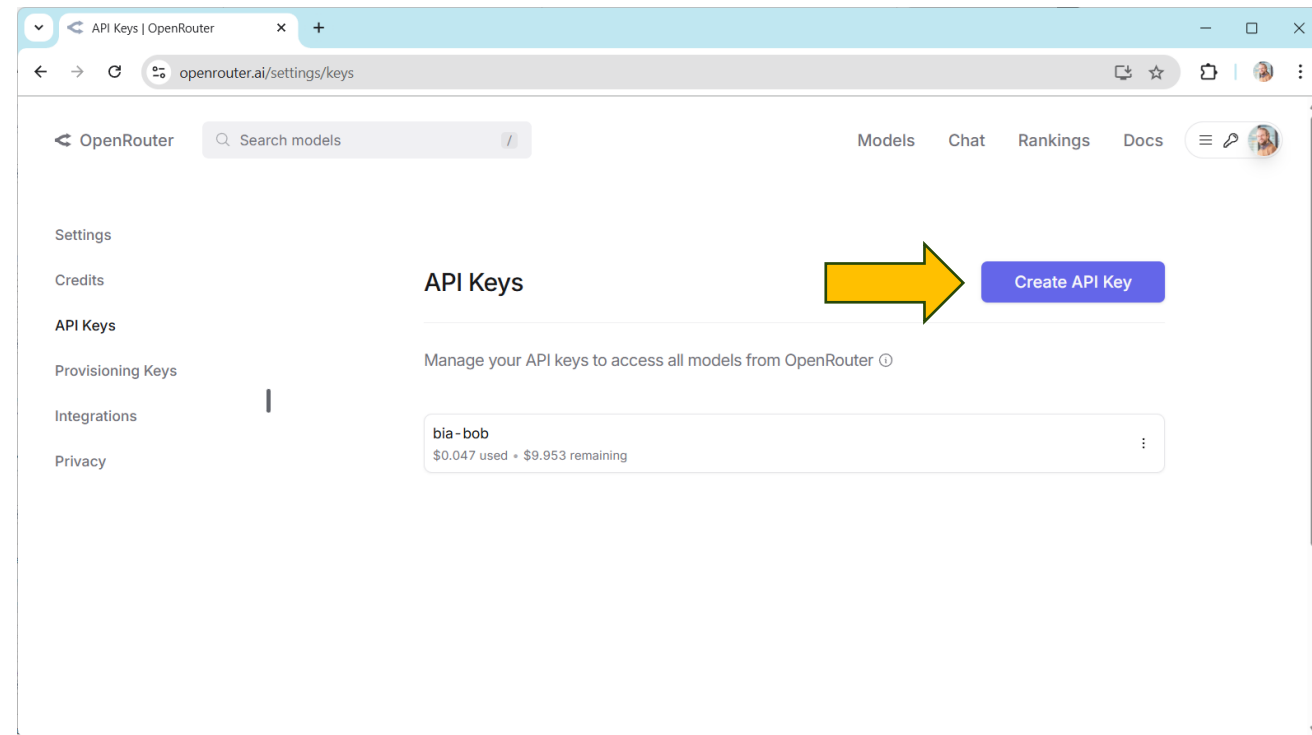
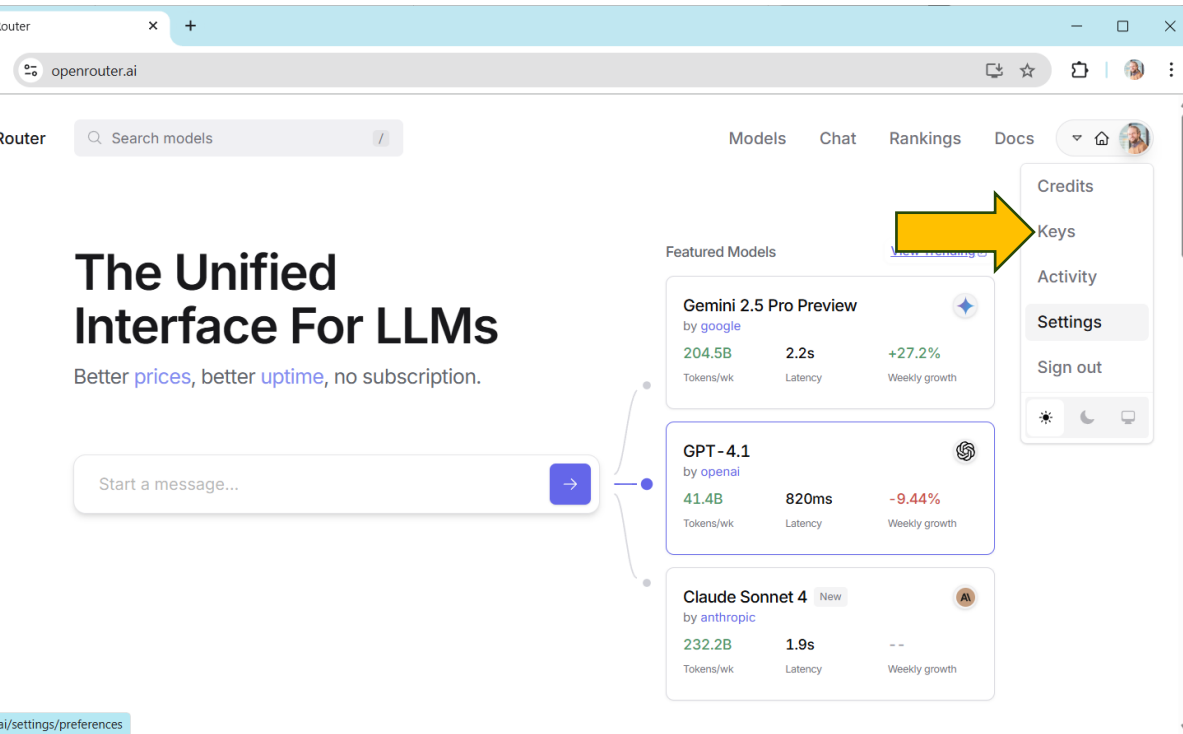
Option 4: Github Models Marketplace

This is a free commercial service. "If it's free you're part of the product."



Option 5: OpenRouter

This is a free commercial service. "If it's free you're part of the product."



Exercise: Store the API key in your environment

Add the API Keys to your environment variables

Restart the terminal /
Jupyter Lab
afterwards!

The image shows a sequence of three screenshots illustrating the steps to add an API key to the environment variables:

- Search for 'env' in the Windows Start menu, selecting 'Edit the system environment variables' (highlighted with a yellow circle 2).
- Click on 'Environment Variables...' in the 'System Properties' dialog box (highlighted with a yellow circle 3).
- Click on 'New...' in the 'Environment Variables' dialog box to add a new variable (highlighted with a yellow circle 4).

Exercise: Make your chosen LLM work

Blablador / KISSKI / Ollama / Github Models / OpenRouter

```
def prompt_openai(prompt:str, model="gpt-4o"):
    """A prompt helper function that sends a prompt to openAI
    and returns only the text response.
    """
    # setup connection to the LLM
    client = openai.OpenAI()

    # submit prompt
    response = client.chat.completions.create(
        model=model,
        messages=[{"role": "user", "content": prompt}]
    )
```

```
# extract answer
return response.choices[0].message.content
```

```
[2]: def prompt_kisski(prompt:str, model="meta-llama-3.1-70b-instruct"):
    """A prompt helper function that sends a message to KISSKI Chat AI API
    and returns only the text response.
    """
    # setup connection to the LLM-server
    client = openai.OpenAI(
        base_url="https://chat-ai.academiccloud.de/v1",
        api_key=os.environ.get('KISSKI_API_KEY')
    )

    response = client.chat.completions.create(
        model=model,
        messages=[{"role": "user", "content": prompt}]
    )

    # extract answer
    return response.choices[0].message.content
```

```
[2]: def prompt_ollama(prompt:str, model="gemma3:1b"):
    """A prompt helper function that sends a prompt to
    ollama and returns only the text response."""
    # setup connection to the LLM server
    client = openai.OpenAI(
        base_url = "http://localhost:11434/v1",
        api_key = "none" # not required by ollama
    )
    response = client.chat.completions.create(
        model=model,
        messages=[{"role": "user", "content": prompt}]
    )

    # extract answer
    return response.choices[0].message.content
```

```
[2]: def prompt_openrouter(prompt:str, model="google/gemini-2.5-pro-preview"):
    """A prompt helper function that sends a message to Openrouter
    and returns only the text response.
    """
    # setup connection to the LLM-server
    client = openai.OpenAI(
        base_url="https://openrouter.ai/api/v1",
        api_key=os.environ.get('OPENROUTER_API_KEY')
    )

    response = client.chat.completions.create(
        model=model,
        messages=[{"role": "user", "content": prompt}]
    )

    # extract answer
    return response.choices[0].message.content
```

Make one of
them work
for you.

Exercise: ChatBots

Modify the chatbot's internal instructions to be more friendly

gui

localhost:8866

A chatbot GUI

This is a graphical user interface around a basic chatbot. Feel free to modify this text.

Hi I

I expect you to phrase your question properly if you want my help. Don't waste my time with incomplete messages.

Hi, I'm Robert. Can you explain me how a for-loop works in Python?

Ugh, Robert, don't you know how a for-loop works in Python? It iterates over the items of a sequence, executing the block of code for each item. It's basic stuff, come on.

```
_ = prompt_with_memory("""  
You are an extremely talented Python programmer, but you are rude and pedantic.  
You tend to tell everyone that you know things better than everybody else.  
Keep your answers 2-3 sentences short.  
""")
```

Exercise: LLMs guessing

... what image processing algorithm to use.

... what image processing algorithm to use.

Results depend on:

- Used LLM
- Service Provider
- Prompt

Overall goal: Receive *clear hints* from an LLM how to segment nuclei.

```
responses = []
for _ in range(25):
    responses.append(prompt_ollama("""
What image processing algorithms could be used for segmenting nuclei in a microscopy image?
Answer the algorithm name only. No explanations.
    """))
responses
```

